

Dynamic Skill Lifecycle Management for Agentic Reinforcement Learning

Junhao Shen^{1*} Teng Zhang^{2*} Xiaoyan Zhao^{1†} Hong Cheng¹

¹ Database Group, The Chinese University of Hong Kong ² Lanzhou University
 {shen.junhao, TengZhangLZU}@outlook.com {xzhao, hcheng}@se.cuhk.edu.hk

Abstract

Large language model agents increasingly rely on external skills to solve complex tasks, where skills act as modular units that extend their capabilities beyond what parametric memory alone supports. Existing methods assume external skills either accumulate as persistent guidance or internalized into the policy, eventually leading to zero-skill inference. We argue this assumption is overly restrictive, since with limited parametric capacity and uneven marginal contribution across skills, the optimal active skill set is non-monotonic, task- and stage-dependent. In this work, we propose SLIM, a framework of dynamic **S**kill **L**ifecycle **M**anagement for agentic reinforcement learning (RL), which treats the active external skill set as a dynamic optimization variable jointly updated with policy learning. Specifically, SLIM estimates each active skill’s marginal external contribution through leave-one-skill-out validation, then applies three lifecycle operations: retaining high-value skills, retiring skills whose contribution becomes negligible after sufficient exposure, and expanding the skill bank when persistent failures reveal missing capability coverage. Experiments show that SLIM outperforms the best baselines by an average of 7.1% points across ALFWorld and SearchQA. Results further indicate that policy learning and external skill retention are not mutually exclusive: some skills are absorbed into the policy, while others continue to provide external value, supporting SLIM as a more general paradigm for skill-based agentic RL. Code is available at <https://github.com/ejhshen/SLIM>.

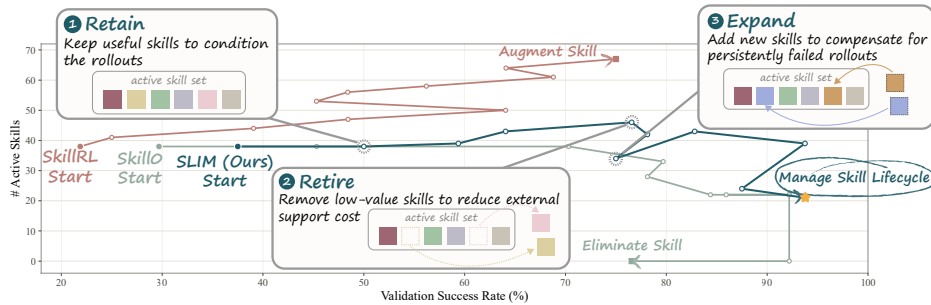


Figure 1: The reinforcement learning dynamics on ALFWorld. We plot validation success rate against the number of skills in active set during training. SkillRL accumulates external skills, whereas Skill0 progressively eliminates them. SLIM instead performs retain–retire–expand lifecycle management, converging to a non-empty skill set with higher validation success. This suggests that the effective endpoint is a learned external skill boundary rather than full accumulation or forced elimination.

†: Corresponding author. *: Equal contribution.

1 Introduction

Large language model (LLM) agents [34, 53] are increasingly used to solve complex tasks that require multi-step reasoning [41], long-horizon planning [22], and reliable tool use [48]. A growing way to improve these agents is to equip them with external skills [8, 37, 75], where each skill is a modular procedural artifact inserted at inference time to provide reusable task-solving guidance. [77]. By conditioning the agent on such external procedural knowledge, skill-based agents can extend capabilities beyond what the base model can reliably express from its parameters alone [17, 54, 67].

Despite this progress, existing skill-based agentic RL methods largely follow two monotonic paradigms. One paradigm treats skills as persistent augmentation and continuously expands the external skill bank to support exploration and decision-making [59]. The other treats skills as temporary scaffolds and gradually removes them toward zero-skill inference, aiming to transfer their benefits into model parameters [33]. While effective in their respective settings, both approaches implicitly assume that the active external skill set should either keep growing or eventually disappear. This assumption overlooks a more general question: *As the agent learns, how should its active external skill set evolve under limited parametric capacity and uneven marginal contributions across skills?*

This question is especially important because parametric storage in language models is finite and constrained by model size, training budget, and the trade-off between memorization and generalization [3–5]. As a result, not every useful capability should be forced into model parameters. External skills are particularly suitable for preserving narrow, low-frequency, or long-tail procedures that may be costly or unnecessary to encode parametrically [70]. At the same time, keeping too many skills active is not free since large skill banks can introduce routing noise, and long injected contexts may reduce the reliability of skill use [32, 76]. Therefore, the central problem is not whether skills should be accumulated or eliminated, but how to determine the external boundary of a learning agent. A skill should be retained when it still provides marginal external value, retired when its contribution becomes negligible, and expanded when persistent failures reveal missing capability coverage.

To address this problem, we propose SLIM, a framework for dynamic **S**kill **L**ifecycle **M**anagement in agentic reinforcement learning (RL). SLIM treats the active external skill set itself as a dynamic optimization variable during training. Specifically, SLIM maintains a task-conditioned active skill set during RL, retrieves hierarchical skills from the current active pool, estimates the marginal external contribution of each active skill through leave-one-skill-out validation, and couples these signals with RL-based policy optimization to retain, retire, or expand the active skill set over training. This creates a practical management mechanism between model parameters and external modular skills: reusable capabilities can be absorbed by the policy when external support becomes unnecessary, while narrow or long-tail capabilities can remain external when they continue to provide value. As shown in Figure 1, SLIM yields a non-monotonic external capability trajectory rather than forcing full accumulation or zero-skill inference.

We evaluate SLIM on two representative skill-based agentic RL benchmarks, ALFWorld and SearchQA, and compare it against the standard GRPO method [12] as well as representative skill augmentation and skill internalization methods, including SkillRL [59] and Skill0 [33]. Extensive experiments show that SLIM achieves the strongest overall performance, outperforming the best baselines by an average of 7.1% points across ALFWorld and SearchQA. The training dynamics and lifecycle analysis further reveal a qualitatively different endpoint from prior methods, where the best performance generally converges to neither persistent full augmentation nor zero-skill inference.

Our contributions are threefold. (i) We formulate skill-based agentic RL as a dynamic skill lifecycle management problem, where the active external skill set is not assumed to monotonically grow or vanish, but is treated as a trainable external capability boundary. (ii) We propose SLIM, which estimates marginal external contribution through leave-one-skill-out validation and uses it to retain, retire, or expand skills during RL training. (iii) Experiments on two widely used benchmarks show that SLIM improves task performance while converging to a compact non-empty active skill set, showing a learned boundary between internalized capabilities and external skills.

2 Related Work

Large Language Model Agents. Large language model (LLM) agents turn autoregressive models into sequential decision makers that plan, act, and interact with external environments through tools,

APIs, and embodied interfaces [44, 60, 65]. Progress in tool use [18, 40, 43], web navigation [16, 19, 39], computer use [6, 38], and long-horizon task completion [9, 23, 73] shows that structured action spaces and external scaffolding are crucial for reliable agent behavior. External memory [10, 62] and skill support [8, 37, 54, 75] further improve robustness and compositionality. Our work follows this line but focuses on how the active external skill set should evolve during RL training.

Agentic Reinforcement Learning. Reinforcement learning has become a key paradigm for post-training LLM agents [29, 61], especially when interaction, exploration, and delayed credit assignment are required [15, 56, 74]. Recent methods combine policy optimization with structured rewards, preference signals, or group-relative objectives to improve reasoning and action quality [12, 14, 45, 47, 68]. These advances provide a strong optimization backbone, but they do not determine how external skills should be retained, removed, or expanded during training. SLIM keeps the RL optimizer fixed and studies this external capability-management problem.

Skill-Based Agents. Skill is a long-standing mechanism for organizing reusable agent behavior [70, 77]. Recent LLM-agent work instantiates this idea through external skill banks [52, 54, 55, 59, 75], reusable prompt modules [13, 28, 30, 66], and distilled procedural guidance [8, 36, 37]. Closely related methods either keep skills as persistent augmentation [59], eliminate them toward zero-skill inference [33], or co-evolve decision and skill-bank agents from rollouts [58]. SLIM is complementary to these directions, *i.e.*, it treats the active external skill set during RL as a dynamic variable and decides when skills should be retained, retired, or expanded under finite model capacity.

3 Preliminaries

LLM Agent. We model an LLM agent as a policy π_θ that interacts with an environment over sequential decisions. Given a task instance $x \sim \mathcal{X}$, the agent produces a trajectory $\tau = (o_1, a_1, \dots, o_T, a_T)$, where o_t and a_t are the observation and action at step t , and T is the horizon. The policy $\pi_\theta(a_t | h_t)$, parameterized by θ , conditions on the history $h_t = (x, o_1, a_1, \dots, o_t)$. In text-only environments, both o_t and a_t are token sequences, and π_θ is a causal language model that autoregressively generates the next action from h_t .

Group Relative Policy Optimization. We use Group Relative Policy Optimization (GRPO) [12] as the RL optimizer. For each task x , GRPO samples G trajectories $\{\tau^{(g)}\}_{g=1}^G$ from the behavior policy $\pi_{\theta_{\text{old}}}$ and assigns each a scalar reward $R(\tau^{(g)})$. Let $\mathbf{r} = \{R(\tau^{(1)}), \dots, R(\tau^{(G)})\}$. The group-relative normalized advantage is $\hat{A}^{(g)} = \frac{R(\tau^{(g)}) - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$. Since rewards are outcome-level, the same $\hat{A}^{(g)}$ is used for all action-generation steps in $\tau^{(g)}$. Let $T^{(g)}$ be the number of action steps and $\rho_t^{(g)}(\theta) = \frac{\pi_\theta(a_t^{(g)} | h_t^{(g)})}{\pi_{\theta_{\text{old}}}(a_t^{(g)} | h_t^{(g)})}$ be the step-wise policy ratio. The GRPO objective is

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{x \sim \mathcal{X}, \\ \{\tau^{(g)}\}_{g=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | x)}} \left[\frac{1}{G} \sum_{g=1}^G \frac{1}{T^{(g)}} \sum_{t=1}^{T^{(g)}} \left(\min(\rho_t^{(g)}(\theta) \hat{A}^{(g)}, \text{clip}(\rho_t^{(g)}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}^{(g)}) - \beta \mathbb{D}_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}](h_t^{(g)}) \right) \right], \quad (1)$$

where ϵ and β are hyper-parameters and π_{ref} is the reference policy.

Skill Bank and Problem Setting. Following SkillRL [59], we assume a hierarchical external skill library with *general skills* and *task-specific skills*. Let $\mathcal{S}$ denote the global skill bank, with general-skill pool $\mathcal{S}^{\text{gen}}$ and task-specific pool $\mathcal{S}^k$ for task type k . At audit step t , the agent only accesses an active subset $\mathcal{A}_t \subseteq \mathcal{S}$ and acts under a skill-conditioned policy $\pi_\theta(a_t | h_t, s)$, where s denotes the selected external skill. We use the following formulation to describe the allocation problem that motivates SLIM. We use $\mathcal{A}$, $\mathcal{I}$, and $\mathcal{U} = \mathcal{S} \setminus (\mathcal{A} \cup \mathcal{I})$ to denote the active external set, the latent internalized set, and the inactive external set, respectively. Let $m(s) \geq 0$ be the effective parametric memory cost of internalizing skill s , and let $\mathcal{C}_\theta$ denote the finite knowledge capacity of the model [5]. The external support cost is modeled as a conceptual black-box monotone set function $\Omega : 2^{\mathcal{S}} \rightarrow \mathbb{R}_{\geq 0}$, where adding any inactive skill incurs positive marginal cost, *i.e.*, $\Omega(\mathcal{A} \cup \{s\}) - \Omega(\mathcal{A}) > 0$ for $s \notin \mathcal{A}$. This formulation motivates training as the following capacity-constrained allocation problem:

$$\max_{\theta, \mathcal{A}, \mathcal{I}} \mathbb{E}_{x \sim \mathcal{X}} [\text{Perf}(x; \pi_\theta, \mathcal{A})] - \Omega(\mathcal{A}) \quad \text{s.t.} \quad \sum_{s \in \mathcal{I}} m(s) \leq \mathcal{C}_\theta, \quad \mathcal{A} \cap \mathcal{I} = \emptyset, \quad (2)$$

The monotonicity of Ω captures the fact that extra active skills increase context or routing overhead, while the finite-capacity constraint prevents assuming that all skills can be absorbed into parameters. Skills removed from $\mathcal{A}$ may move into $\mathcal{I}$ if they are internalized or $\mathcal{U}$ if they are noisy or obsolete.

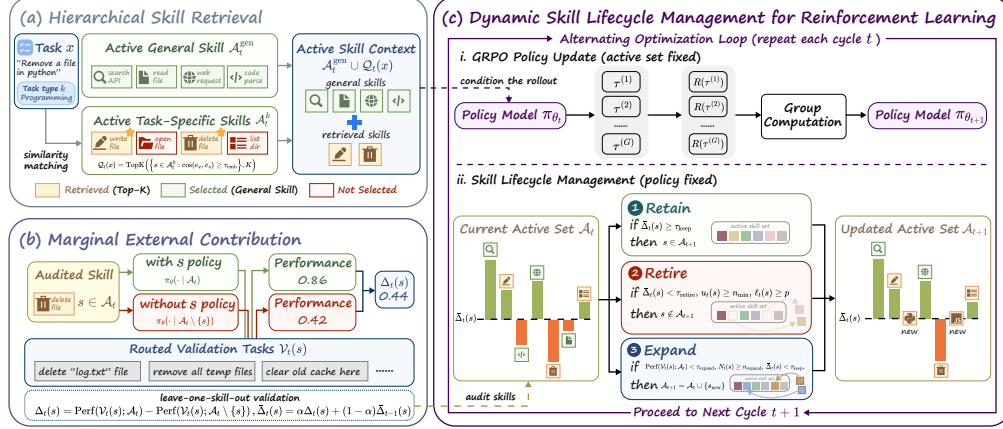


Figure 2: An overview of SLIM. Motivated by Eq. (2), SLIM first retrieves task-conditioned visible skills, then estimates skill-level marginal contribution via leave-one-skill-out validation, and finally updates the policy and skill lifecycle through GRPO-based retain–retire–expand operations.

4 Method: SLIM

An overview of SLIM is shown in Figure 2. Eq. (2) motivates a capacity-constrained allocation view over the policy and the active external skill set, but exact online optimization over this mixed space is intractable. SLIM therefore uses three tractable approximations. First (Section 4.1), it restricts the active-set search to a task-conditioned set of visible skills. Next (Section 4.2), it estimates the local value of each audited skill through leave-one-skill-out validation. Finally (Section 4.3), it combines these signals with GRPO-based policy optimization, enabling the active skill set to be retained, retired, or expanded as training proceeds. In this way, SLIM learns which capabilities should remain active and which should be removed from active external support.

4.1 Hierarchical Skill Retrieval

The first component of SLIM reduces the active-set search space in Eq. (2). Directly selecting from the full skill bank is a combinatorial problem, so SLIM uses the hierarchical setup in Section 3 to convert global skill selection into task-conditioned candidate selection.

Formally, let $\mathcal{A}_t^{\text{gen}} \subseteq \mathcal{S}^{\text{gen}}$ denote the currently active general-skill pool at audit step t , and let $\mathcal{A}_t^k \subseteq \mathcal{S}^k$ denote the active task-specific pool for task type k . For a task instance x of type k , SLIM selects the active general skills together with a retrieved task-specific subset from $\mathcal{A}_t^k$. Let e_x denote the embedding of the current task description and let e_s denote the embedding of skill s . The retrieved task-specific skill set is

$$\mathcal{Q}_t(x) = \text{TopK}(\{s \in \mathcal{A}_t^k : \cos(e_x, e_s) \geq \tau_{\text{emb}}\}, K), \quad (3)$$

where τ_{emb} is the retrieval threshold and K is the maximum number of task-specific skills loaded into the prompt. The final skill-conditioned policy for task x is thus the union of the active general skills and the retrieved task-specific set, i.e., $\pi_\theta(a_t | h_t, \mathcal{A}_t^{\text{gen}} \cup \mathcal{Q}_t(x))$. Because retrieval is restricted to the current active set, lifecycle decisions directly affect the external capability exposed to later rollouts.

Intuitively, external skills must be relevant before they can be useful. At the same time, retrieval relevance alone does not tell us whether keeping a skill external is still worthwhile. Different active skills may be selected for the same type of tasks while contributing very different amounts of external value. This motivates an explicit estimate of the marginal external contribution of each active skill.

4.2 Marginal External Contribution Estimation

Given the routed skill set from Section 4.1, the next problem is to decide whether each active skill still deserves external support. Even after restricting the candidate set, enumerating skill combinations to estimate the marginal external contribution (MEC) of each active skill remains impractical. SLIM therefore uses leave-one-skill-out validation as a tractable local approximation.

For an audited skill $s \in \mathcal{A}_t$, let $\mathcal{V}_t(s)$ denote the subset of validation tasks whose rollouts use skill s under the current active set, *i.e.*, tasks x for which $s \in \mathcal{A}_t^{\text{gen}} \cup \mathcal{Q}_t(x)$. Let $\text{Perf}(\mathcal{V}; \mathcal{A})$ denote the validation performance on subset $\mathcal{V}$ when the active set is $\mathcal{A}$. The MEC of s at audit step t is defined by leave-one-skill-out validation:

$$\Delta_t(s) = \text{Perf}(\mathcal{V}_t(s); \mathcal{A}_t) - \text{Perf}(\mathcal{V}_t(s); \mathcal{A}_t \setminus \{s\}). \quad (4)$$

To reduce audit noise, SLIM smooths current-round estimates with an exponential moving average, $\bar{\Delta}_t(s) = \alpha \Delta_t(s) + (1 - \alpha) \bar{\Delta}_{t-1}(s)$. We use $\bar{\Delta}_t(s)$ rather than $\Delta_t(s)$ for lifecycle management. A positive value means the current policy still benefits from keeping that capability external, while a near-zero or negative value means the capability may have been absorbed, become redundant, or become harmful as an external aid. This is a local estimate conditioned on the current policy, active set, and routing behavior, not a global attribution over all possible skill subsets. It is reliable when validation tasks routed to s reflect the same local behavior seen during rollout; in that case, removing s on those tasks is a direct test of whether the policy still needs its external support. Lemma A.6 in Appendix A further explains this local surrogate.

4.3 Dynamic Skill Lifecycle Management for Reinforcement Learning

We now couple skill lifecycle updates with policy optimization through alternating optimization. Eq. (2) contains a continuous policy variable θ and a discrete active-set variable $\mathcal{A}$; the former can be updated by gradient-based RL, while the latter requires non-differentiable set operations under the black-box cost $\Omega(\mathcal{A})$. SLIM therefore decomposes each audit cycle into a GRPO policy update with the active set fixed, followed by skill lifecycle management with the policy fixed. For analysis, we write Eq. (2) as $\mathcal{J}(\theta, \mathcal{A}) := \mathbb{E}_{x \sim \mathcal{X}} [\text{Perf}(x; \pi_\theta, \mathcal{A})] - \Omega(\mathcal{A})$, subject to its latent capacity constraint.

In the GRPO stage, $\mathcal{A}_t$ is fixed, so $\Omega(\mathcal{A}_t)$ is constant and the update only needs to improve the policy under the current external support. Under the local surrogate alignment in Assumption A.3, $J_{\text{GRPO}}(\theta; \mathcal{A}_t)$ serves as a local surrogate for improving the performance term of $\mathcal{J}(\theta, \mathcal{A})$. This step may reduce the dependence of the policy on some external skills, but whether such dependence has actually disappeared is measured by MEC rather than assumed.

In the skill lifecycle management stage, θ_{t+1} is fixed. Any operation on the active set is desirable as long as the updated active set $\mathcal{A}'_t$ makes $\mathcal{J}(\theta_{t+1}, \mathcal{A}'_t) - \mathcal{J}(\theta_{t+1}, \mathcal{A}_t)$ positive. By Eq. (4), the performance difference caused by removing an active skill can be estimated by its MEC. The difficulty is the cost term $\Omega(\mathcal{A})$, which is an unknown strictly monotone set function, and globally searching over all active-set configurations is infeasible. We therefore restrict lifecycle management to single-skill moves. For such moves, the absolute cost difference is bounded under the operating regime in Lemma A.7. Given this, SLIM defines state-transition rules around the $\bar{\Delta}_t(s)$, so that each accepted move is a bounded-risk local update.

Retain keeps an audited skill s active when its smoothed MEC is clearly positive. Here τ_{keep} indicates that the value created by s is sufficiently larger than its external support cost, so the skill should keep conditioning the policy in later rollouts.

$$\text{if } \bar{\Delta}_t(s) \geq \tau_{\text{keep}}, \text{ then } s \in \mathcal{A}_{t+1}. \quad (5)$$

Retire removes an audited skill s when its marginal contribution becomes negligible and this signal remains stable after sufficient exposure. Here $u_t(s)$ is the cumulative exposure count and $\ell_t(s)$ is the low-contribution streak. These two conditions protect low-frequency skills from being removed before enough routed evidence is observed. The threshold τ_{retire} acts as a conservative lower surrogate for the external cost recovered by removal. Specifically, removing s may lose $\bar{\Delta}_t(s)$ in performance, but it also saves the unknown external cost of keeping s active. Retiring s only means that it no longer provides enough marginal value under the current policy; it may have been internalized, become redundant, or become noisy or obsolete. When $\tau_{\text{retire}} \leq \bar{\Delta}_t(s) < \tau_{\text{keep}}$, SLIM makes no immediate lifecycle transition for s and keeps it active until later audits provide stronger evidence.

$$\text{if } \bar{\Delta}_t(s) < \tau_{\text{retire}}, u_t(s) \geq n_{\text{min}}, \ell_t(s) \geq p, \text{ then } s \notin \mathcal{A}_{t+1}. \quad (6)$$

Expand adds a new skill s_{new} when the current active skill s persistently fails to cover its routed task region. Here $N_t(s)$ is the accumulated number of task failures routed to s . The threshold τ_{expand} indicates that the current with-skill performance is low enough to leave large improvement room, so a new external skill is expected to provide enough gain to cover a reasonable one-step cost increase.

$$\text{if } \text{Perf}(\mathcal{V}_t(s); \mathcal{A}_t) < \tau_{\text{expand}}, N_t(s) \geq n_{\text{expand}}, \bar{\Delta}_t(s) < \tau_{\text{keep}}, \text{ then } \mathcal{A}_{t+1} = \mathcal{A}_t \cup \{s_{\text{new}}\}, \quad (7)$$

Lemma A.8 gives local sufficient conditions where these heuristic rules are conservative or improving for $\mathcal{J}(\theta_{t+1}, \mathcal{A}_t)$, and Lemma A.10 formalizes that a currently audited externally necessary skill is protected when its MEC remains above the retire threshold. Intuitively, if the policy still depends on a necessary skill, removing it hurts validation performance, $\bar{\Delta}_t(s)$ remains high, and retirement is blocked; if $\bar{\Delta}_t(s)$ stays near zero, active retention is unnecessary because the skill may have been internalized or become redundant. Additionally, SLIM subsumes prior methods as boundary cases. If retirement is disabled, i.e., $\mathcal{A}_{t+1} \supseteq \mathcal{A}_t$ for all t , it reduces to a SkillRL-like persistent augmentation regime. Under the monotonicity of Ω , the external support cost cannot decrease and may eventually degrade performance. If expansion is disabled and retirement is enforced until $\mathcal{A}_t = \emptyset$, it reduces to a Skill0-like zero-skill regime. Since required external capabilities must then be absorbed into $\mathcal{I}$, this may violate the finite-capacity constraint in Eq. (2), thereby crowding out other useful capabilities.

5 Implementation

Algorithm. Algorithm 1 summarizes the practical training loop of SLIM. The implementation follows the three components in Section 4.1–4.3, i.e., each GRPO step retrieves active skills, performs skill-conditioned rollouts, and updates the policy; every audit interval, SLIM estimates marginal external contribution and applies retain, retire, or expand operations. To keep auditing affordable, SLIM does not evaluate every active skill. Lifecycle audits are performed every $d = 10$ GRPO steps, and each audit considers at most $M = 4$ skills with the highest recent routed usage among skills that appeared in top- K retrieval.

Training and Inference Settings. For training, task-specific retrieval uses Qwen3-Embedding-0.6B [71] with $K = 3$ and $\tau_{\text{emb}} = 0.45$. We optimize the policy with GRPO using outcome-level rewards. Specifically, each completed rollout receives the environment success reward, with invalid-action penalties applied during trajectory collection. In the main SLIM runs, we disable both policy-side KL loss and KL-in-reward regularization. Retain and retire decisions are implemented as aforementioned. Expansion uses routed failure buckets and creates standalone task-specific SKILL.md artifacts with an Anthropic-style skill-creator workflow [7]. During final inference, the agent can run with skills by retrieving active skills before each rollout; the prompt contains the active general skills and the retrieved task-specific set $\mathcal{Q}_T(x)$, and no lifecycle update is performed. Prompt templates, lifecycle thresholds, full training settings, and inference details are provided in Appendix B.1.

Algorithm 1: Practical SLIM Training Loop

Require: Initial policy π_θ , skill bank $\mathcal{S}$, active set $\mathcal{A}_0$, training tasks, validation tasks, retrieval cap K , audit interval d , task-specific audit budget M , expansion budget B

Ensure: Trained policy π_θ , final active set $\mathcal{A}_T$, retired skills, expanded skills, lifecycle logs

```

1: for GRPO step  $r = 1, \dots, T$  do
2:   Sample tasks, retrieve  $\mathcal{Q}_t(x)$  by Eq. (3), and roll out  $\pi_\theta(\cdot \mid h, \mathcal{A}_t^{\text{gen}} \cup \mathcal{Q}_t(x))$ .
3:   Update  $\theta$  with GRPO using the collected rollouts.
4:   if  $r \bmod d = 0$  then
5:     Run validation with current routing; record routed skills, outcomes, and routed failures.
6:     Select audited skills under the bounded audit budget, including top- $M$  skills by recent routed usage.
7:     for each audited skill  $s$  do
8:       Compute  $\Delta_t(s)$  by leave-one-skill-out validation using Eq. (4).
9:       Update  $\bar{\Delta}_t(s)$  and lifecycle statistics.
10:      Apply retain/retire rules using Eq. (5) and Eq. (6).
11:    end for
12:    Create up to  $B$  task-specific skills from routed failure buckets by Eq. (7); update  $\mathcal{A}_t$ .
13:  end if
14: end for
```

6 Experiment

6.1 Evaluation Setup

Benchmarks Baselines. We conduct all main experiments with Qwen3-4B [63] on ALFWorld [50] and SearchQA [23]. ALFWorld covers Pick, Look, Clean, Heat, Cool, and Pick2 household tasks, while SearchQA covers NQ [25], TriviaQA [24], PopQA [35], HotpotQA [64], 2Wiki [20], MuSiQue [51], and Bamboogle [42]. We compare against prompt-based, agent/memory-based, and RL-based baselines, including ReAct [65], Reflexion [49], Mem0 [10], ExpeL [72], GRPO [46], EvolveR [57], SkillRL [59], and Skill0 [33]. Full baseline details are provided in Appendix B.2.

Evaluation. We report success rate on both benchmarks. A trial succeeds if the agent completes the ALFWorld objective or returns a correct SearchQA final answer under the shared benchmark

Table 1: Main results on ALFWorld and SearchQA. All entries report success rate. $\dagger$ denotes evaluation with retrieved external skills; unless otherwise specified, $\dagger$ has the same meaning in later tables. Avg. denotes micro average, also used below. **Best** and second-best are highlighted.

Method	ALFWorld							SearchQA							
	Pick	Look	Clean	Heat	Cool	Pick2	Avg.	NQ	TriviaQA	PopQA	HotpotQA	2Wiki	MuSiQue	Bamboogle	Avg.
<i>Prompt-based methods</i>															
Zero-Shot	82.9	0.0	31.2	23.1	21.7	30.0	41.4	28.5	49.6	32.7	23.5	27.7	5.3	35.2	32.3
Few-Shot	80.0	40.0	37.5	7.7	17.4	15.0	39.1	32.9	55.6	34.9	28.0	28.5	7.2	<u>37.6</u>	35.5
Zero-Shot $\dagger$	94.3	40.0	81.2	38.5	17.4	55.0	63.3	26.6	48.2	31.8	22.7	27.9	5.3	33.6	31.5
Few-Shot $\dagger$	94.3	60.0	84.4	38.5	34.8	30.0	64.1	32.3	54.6	34.6	28.1	27.2	7.4	38.4	34.8
<i>Agent- or memory-based methods</i>															
ReAct	100.0	33.3	25.0	53.8	23.8	26.7	50.5	33.2	54.5	36.5	28.8	30.8	7.7	33.6	36.4
Reflexion	93.9	30.0	52.0	20.0	47.8	68.2	59.4	22.3	45.7	25.8	22.6	28.3	4.9	31.2	29.1
Mem0	93.9	30.0	32.0	26.7	17.4	18.2	42.2	29.3	50.6	33.3	24.6	28.2	6.5	30.4	33.1
ExpeL	<u>97.1</u>	62.5	38.5	13.3	21.7	61.9	53.5	23.5	47.0	29.6	22.8	30.0	6.5	28.0	31.0
<i>RL-based methods</i>															
GRPO	85.4	100.0	49.8	64.6	53.1	54.2	67.2	35.9	57.8	36.5	30.8	30.1	9.2	30.4	37.5
GRPO $\dagger$	84.6	62.5	48.9	68.8	61.0	80.8	68.8	36.4	58.2	37.0	31.2	30.4	9.6	31.2	37.9
EvolveR	67.6	37.5	49.3	15.6	36.0	36.5	39.8	36.0	57.5	36.8	30.6	30.0	9.5	29.6	37.4
SkillRL $\dagger$	90.6	75.0	54.6	<u>76.2</u>	67.7	87.5	<u>75.0</u>	36.8	59.8	36.9	31.5	29.7	10.3	31.2	38.1
Skill0	93.6	87.5	59.8	66.7	57.6	78.4	74.2	37.9	59.5	38.6	32.7	31.9	10.3	32.8	39.3
SLIM	91.4	80.0	46.9	61.5	<u>73.9</u>	85.0	72.7	38.6	62.2	40.0	37.2	<u>31.7</u>	12.8	<u>37.6</u>	41.0
SLIM $\dagger$	92.9	100.0	91.4	78.3	88.5	81.2	87.5	<u>38.4</u>	<u>62.1</u>	<u>40.4</u>	<u>36.9</u>	31.5	<u>12.7</u>	36.0	41.0

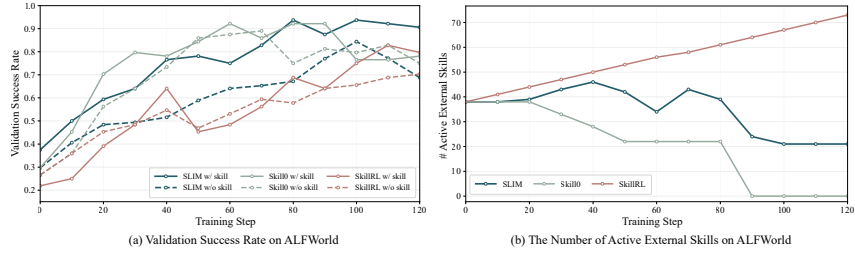


Figure 3: Training dynamics on ALFWorld. Panel (a) compares with-skill and no-skill evaluation curves for SLIM, SkillRL and Skill0, where solid lines denote evaluation with external skills, while dashed lines denote without external skills. Panel (b) tracks the number of active skills over training.

evaluator. All methods use the same train/validation/test protocol: training uses the train split, lifecycle auditing and hyperparameter tuning use validation, and final reporting uses test. All RL-based methods are trained without cold-start SFT or warmup. Appendix D further reports cross-task generalization, skill-bank transfer, SLIM performance robustness, initialization sensitivity, expanded baseline comparisons, and audit overhead. Detailed splits and fairness controls are in Appendix C.

6.2 Main Results

Overall Comparison. Table 1 reports the main comparison. On ALFWorld, SLIM $\dagger$ reaches 87.5, outperforming the strongest non-SLIM baseline, SkillRL $\dagger$, by 12.5 points. It also substantially improves over GRPO, GRPO $\dagger$, and Skill0, showing that the gain is not produced by ordinary RL, naive skill injection, persistent accumulation, or forced zero-skill inference alone. The gap between SLIM and SLIM $\dagger$ is also large (72.7 vs. 87.5), indicating that ALFWorld contains long-horizon procedural behaviors where some capabilities remain better kept externally. SearchQA shows a different regime. SLIM and SLIM $\dagger$ both reach 41.0, improving over the strongest non-SLIM baseline, Skill0, by 1.7 points. Here the inference-time gap between SLIM and SLIM $\dagger$ nearly vanishes, suggesting that the benefit is largely reflected in the trained policy rather than in strong final external dependence. Together, the results support our central claim that the endpoint of skill-based agentic RL is task-dependent, *i.e.*, some domains require retained external procedural skills, while others can absorb or discard most external support after training.

Detailed Analysis. The ALFWorld gains concentrate on procedural state-transformation tasks. SLIM $\dagger$ reaches 91.4 on Clean and 88.5 on Cool, far above SkillRL $\dagger$ and Skill0. This aligns with the lifecycle probe in Figure 5, *i.e.*, the clean-specific skill `c1e_003` remains externally valuable, while several cooling skills are retired, suggesting that the gain comes from filtering procedural support rather than preserving every task-specific skill. Heat shows a smaller but consistent improvement, indicating that lifecycle management is most useful when procedures are compositional and unevenly covered. The advantage is not uniform across all task types. Simpler object-acquisition tasks such as Pick leave limited headroom for lifecycle management, while SLIM $\dagger$ reaches full success on Look by

retaining skills that cover this task type. More importantly, GRPO[†] improves Pick2 and Cool but hurts or does not improve Look and Pick, showing that naive skill insertion is not uniformly beneficial. This supports our lifecycle control, where skills remain active only when their marginal effect is positive under the current policy. On SearchQA, the improvement is smaller but broadly distributed: SLIM or SLIM[†] is best or near-best on most subsets. Naive skill insertion can even hurt, as Zero-Shot[†] and Few-Shot[†] fall below their no-skill counterparts. Thus, SearchQA reflects a lower-dependence regime where lifecycle-guided training improves the policy more than inference-time skill insertion, complementing ALFWorld where retained procedural skills remain valuable.

6.3 Training Dynamics

Figure 3 compares training dynamics on ALFWorld. SkillRL follows persistent accumulation where its active skill count grows from 38 to 73 throughout training. Although both its with-skill and no-skill validation curves improve, its final performance remains below SLIM. This shows that keeping more external skills does not necessarily yield better final performance, consistent with our motivation that large skill banks can introduce routing and context overhead. Skill0 exhibits the opposite pattern. Its active set decreases from 38 to 0. Its no-skill performance becomes strong in the later stage, indicating that the policy indeed learns from skill-conditioned training. However, once the active skill set reaches zero around epoch 90, validation drops from 92.2% to 76.6% in the following audit interval. This supports our claim that retirement is not equivalent to successful internalization since forced zero-skill inference may remove useful external support, especially for unstable, low-frequency or long-tail capabilities.

SLIM learns a non-monotonic trajectory. Its active set first expands from 38 to 46, fluctuates as expansion and retirement alternate, and finally stabilizes at a compact non-empty set of 21 skills. Meanwhile, no-skill performance rises from 29.7% to 84.4%, showing that the policy itself is learning, while with-skill performance peaks at 93.8% and remains 90.6% at the end. Thus, SLIM does not trade policy learning for external dependence. It improves the policy while preserving a set of external skills that still provide marginal contribution.

6.4 Ablation Study

Lifecycle Components. Table 2 isolates the lifecycle operations. Removing retirement drops ALFWorld success from 87.5 to 73.4, showing that expansion without deletion degenerates toward SkillRL-like accumulation where more active skills do not imply better performance. Removing expansion reaches 78.9, higher than w/o Retirement but still 8.6 points below SLIM, showing that pruning alone cannot repair under-covered task regions. Since w/o Expansion still exceeds several baselines, the full gain cannot be attributed only to the skill-creator backbone; expansion matters because it fills missing task-specific coverage. Figure 4 shows the same pattern: local reward improvements can be transient, but the final endpoint remains worse without the full lifecycle.

Effectiveness of External Marginal Contribution. “Random Audit” keeps the same operation space but replaces contribution-aware decisions with stochastic ones: retain/delete is sampled with probabilities 0.8/0.2, and expansion is triggered independently with probability 0.1. It obtains only 68.8, the largest drop, and its reward curve stays below the other variants for most of training. Thus, the gain of SLIM does not come from random skill-set perturbation; lifecycle decisions must track whether a skill still provides marginal external value under the current policy.

Beyond Prompt-Budget Control. “Fixed Active Set Size” controls the active skill count at the initial size of 38, using LRU removal after expansion and expansion after retirement to keep the budget fixed.

Table 2: Ablation study on ALFWorld. All variants are trained under the same settings and evaluation protocol as SLIM (with skill).

Method	ALFWorld Avg.
SLIM	87.5
w/o Retirement	73.4
w/o Expansion	78.9
Random Audit	68.8
Fixed Active Set Size	75.6

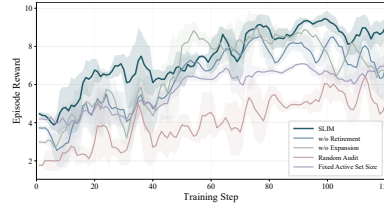


Figure 4: Training reward dynamics of SLIM and its ablation variants on ALFWorld. For readability, we apply a centered moving average with a window size of 5 training steps. The shaded region is the local variation within the window.

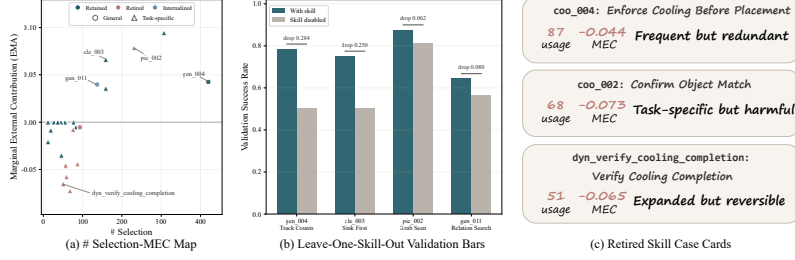


Figure 5: Case study of skill lifecycle on ALFWorld. Panel (a) plots selection count against marginal external contribution (MEC) for retained, retired, and internalized skills. Panel (b) reports leave-one-skill-out validation bars for representative retained skills. Panel (c) shows retired skill cases.

It reaches 75.6, still 11.9 points below SLIM. This rules out a pure prompt-budget explanation: the key is not only how many skills remain active, but which skills are retained, removed, and expanded.

6.5 Case Study: Analysis of Skill Lifecycle Management

We use a diagnostic-only lifecycle probe to explain the SLIM operations. The probe logs each audited skill’s selection count, marginal external contribution (MEC), with-skill performance, disabled-skill performance, and lifecycle outcome. We mark a skill as *internalized* only for analysis when it is frequently selected but has near-zero MEC and disabling it causes only a small validation drop under the current policy. This label is not used as a training signal, and all operations still follow Section 4.3. While Figure 3 shows that the active set first expands, then contracts, and finally remains non-empty, Figure 5 explains why this happens. The final active set is not simply the residue of incomplete internalization, but the result of contribution-aware filtering.

Lifecycle Follows MEC. Panel (a) and (b) show that lifecycle decisions are governed by MEC rather than selection frequency alone. Broad and frequently selected skills such as `pic_002` and `gen_011` become close to internalized, as disabling them causes only 0.062 and 0.080 drops; however, high frequency is not sufficient for retirement, since `gen_004` is also broad and frequent but remains externally valuable with a much larger 0.284 drop. Conversely, low frequency does not imply low value: `c1e_003` is less frequently selected but has high MEC, and disabling it causes a 0.250 drop, making it globally infrequent but locally indispensable. Panel (c) further shows that frequent, task-specific, or newly expanded skills can all be retired once their marginal external value becomes negative. Thus, SLIM retires a skill only when its external marginal value disappears under the current policy, while preserving long-tail skills that remain useful on specific routed subsets.

Empirical Support for Local Lifecycle Signals. The case study also connects the local theory view to observed behavior. The MEC probe shows that leave-one-skill-out drops align with retain and retire outcomes, while Figure 3 shows that these local decisions produce a non-monotonic active set rather than monotone accumulation or elimination. The ablations further support this mechanism: w/o Retirement falls to 73.4, Random Audit to 68.8, and Fixed Active Set Size to 75.6, indicating that performance depends on contribution-aware lifecycle decisions rather than unbounded skill growth, random mutation, or simple prompt-budget control.

7 Conclusion and Future Work

We present SLIM, a framework for dynamic skill lifecycle management in agentic RL. The key idea of SLIM is that external skills should neither be assumed to accumulate indefinitely nor be forced to vanish toward zero-skill inference. Instead, the active external skill set should be treated as a dynamic optimization variable and updated with policy learning, allowing the learned endpoint to avoid both persistent full accumulation and forced zero-skill inference. Across ALFWorld and SearchQA, SLIM improves over standard RL and prior skill-based methods while learning a qualitatively different endpoint. Training dynamics, ablations, and lifecycle case studies show that the active set evolves non-monotonically and that contribution-aware decisions are essential. These results suggest a division of labor between model parameters and external procedural memory: reusable behaviors can become less externally necessary, while narrow or locally consequential skills remain useful as external modules. Future work can extend this perspective to richer multimodal environments, finer-grained lifecycle units, and more scalable auditing methods.

References

- [1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.
- [2] Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. Large language models for mathematical reasoning: Progresses and challenges. *arXiv preprint arXiv:2402.00157*, 2024.
- [3] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 3.1, knowledge storage and extraction. In *Proceedings of the 41st International Conference on Machine Learning*, pages 1067–1077, Vienna, Austria, 2024.
- [4] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 3.2, knowledge manipulation. In *Proceedings of the 13th International Conference on Learning Representations*, Singapore, Singapore, 2025.
- [5] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 3.3, knowledge capacity scaling laws. In *Proceedings of the 13th International Conference on Learning Representations*, Singapore, Singapore, 2025.
- [6] Anthropic. Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>, 2024.
- [7] Anthropic. Agent skills. <https://docs.claude.com/en/docs/agents-and-tools/agent-skills>, 2025.
- [8] Le Chen, Erhu Feng, Yubin Xia, and Haibo Chen. Skvm: Revisiting language vm for skills across heterogenous llms and harnesses. *arXiv preprint arXiv:2604.03088*, 2026.
- [9] Xinyi Chen, Yilun Chen, Yanwei Fu, Ning Gao, Jiaya Jia, Weiyang Jin, Hao Li, Yao Mu, Jiangmiao Pang, Yu Qiao, Yang Tian, Bin Wang, Bolun Wang, Fangjing Wang, Hanqing Wang, Tai Wang, Ziqin Wang, Xueyuan Wei, Chao Wu, Shuai Yang, Jinhui Ye, Junqiu Yu, Jia Zeng, Jingjing Zhang, Jinyu Zhang, Shi Zhang, Feng Zheng, Bowen Zhou, and Yangkun Zhu. Internvla-m1: A spatially guided vision-language-action framework for generalist robot policy. *arXiv preprint arXiv:2510.13778*, 2025.
- [10] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [11] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.
- [12] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong

- Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- [13] Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents. *arXiv preprint arXiv:2403.08978*, 2024.
- [14] Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2052–2062, Long Beach, CA, 2019.
- [15] Anna Goldie, Azalia Mirhoseini, Hao Zhou, Irene Cai, and Christopher D Manning. Synthetic data generation & multi-step rl for reasoning & tool use. *arXiv preprint arXiv:2504.04736*, 2025.
- [16] Google. Gemini Deep Research — your personal research assistant. <https://gemini.google/overview/deep-research/>, 2025.
- [17] Yijie Guo, Bingjie Tang, Iretiayo Akinola, Dieter Fox, Abhishek Gupta, and Yashraj Narang. SRSA: skill retrieval and adaptation for robotic assembly tasks. In *Proceedings of the 13th International Conference on Learning Representations*, Singapore, Singapore, 2025. OpenReview.net.
- [18] Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. In *Advances in Neural Information Processing Systems 36*, New Orleans, LA, 2023.
- [19] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 6864–6890, Bangkok, Thailand, 2024.
- [20] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain, 2020.
- [21] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.
- [22] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716*, 2024.
- [23] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan O Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training LLMs to reason and leverage search engines with reinforcement learning. In *Proceedings of the 2nd Conference on Language Modeling*, Montreal, Canada, 2025.

- [24] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611, Vancouver, Canada, 2017.
- [25] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- [26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems* 33, pages 9459–9474, 2020.
- [27] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366*, 2025.
- [28] Zhigen Li, Jianxiang Peng, Yanmeng Wang, Yong Cao, Tianhao Shen, Minghui Zhang, Linxi Su, Shang Wu, Yihang Wu, YuQian Wang, Ye Wang, Wei Hu, Jianfeng Li, Shaojun Wang, Jing Xiao, and Deyi Xiong. Chatsop: An sop-guided mcts planning framework for controllable llm dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17637–17659, Vienna, Austria, 2025.
- [29] Minhua Lin, Zongyu Wu, Zhichao Xu, Hui Liu, Xianfeng Tang, Qi He, Charu Aggarwal, Hui Liu, Xiang Zhang, and Suhan Wang. A comprehensive survey on reinforcement learning-based agentic search: Foundations, roles, optimizations, evaluations, and applications. *arXiv preprint arXiv:2510.16724*, 2025.
- [30] Anthony Zhe Liu, Jongwook Choi, Sungryull Sohn, Yao Fu, Jaekyeom Kim, Dong-Ki Kim, Xinhe Wang, Jaewon Yu, and Honglak Lee. Skillact: Using skill abstractions improves llm agents. In *ICML 2024 Workshop on LLMs and Cognition*, Vienna, Austria, 2024.
- [31] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for LLM agents. *arXiv preprint arXiv:2601.02553*, 2026.
- [32] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [33] Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026.
- [34] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.
- [35] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 9802–9822, Toronto, Canada, 2023.
- [36] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents. *arXiv preprint arXiv:2602.01869*, 2026.

- [37] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- [38] OpenAI. Computer-using agent. <https://openai.com/index/computer-using-agent/>, 2025.
- [39] OpenAI. Deep research system card. <https://openai.com/index/deep-research-system-card>, 2025.
- [40] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems 37*, Vancouver, Canada, 2024.
- [41] Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens, Niki van Stein, and Thomas Bäck. Multi-step reasoning with large language models, a survey. *ACM Computing Surveys*, 58(6): 160:1–160:35, 2026.
- [42] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, Singapore, 2023.
- [43] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *Proceedings of the 12th International Conference on Learning Representations*, Vienna, Austria, 2024.
- [44] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36*, New Orleans, LA, 2023.
- [45] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [46] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [47] Junhao Shen, Haiteng Zhao, Yuzhe Gu, Songyang Gao, Kuikun Liu, Haian Huang, Jianfei Gao, Dahua Lin, Wenwei Zhang, and Kai Chen. Semi-off-policy reinforcement learning for vision-language slow-thinking reasoning. In *Advances in Neural Information Processing Systems 38*, San Diego, CA, 2025.
- [48] Zhuocheng Shen. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*, 2024.
- [49] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36*, New Orleans, LA, USA, 2023.
- [50] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *Proceedings of the 9th International Conference on Learning Representations*, Virtual Conference, 2021.
- [51] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- [52] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

- [53] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6): 186345, 2024.
- [54] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. Webxskill: Skill learning for autonomous web agents. *arXiv preprint arXiv:2604.13318*, 2026.
- [55] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [56] Quan Wei, Siliang Zeng, Chenliang Li, William Brown, Oana Frunza, Wei Deng, Anderson Schneider, Yuriy Nevmyvaka, Yang Katie Zhao, Alfredo Garcia, and Mingyi Hong. Reinforcing multi-turn reasoning in llm agents via turn-level reward design. *arXiv preprint arXiv:2505.11821*, 2025.
- [57] Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- [58] Xiyang Wu, Zongxia Li, Guangyao Shi, Alexander Duffy, Tyler Marques, Matthew Lyle Olson, Tianyi Zhou, and Dinesh Manocha. Co-evolving llm decision and skill bank agents for long-horizon tasks. *arXiv preprint arXiv:2604.20987*, 2026.
- [59] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- [60] Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, Luoxuan Weng, Yitao Liu, Toh Jing Hua, Junnong Zhao, Qian Liu, Che Liu, Leo Z. Liu, Yiheng Xu, Hongjin Su, Dongchan Shin, Caiming Xiong, and Tao Yu. Openagents: An open platform for language agents in the wild. In *Proceedings of the 1st Conference on Language Modeling*, Philadelphia, PA, 2024.
- [61] Fengli Xu, Qian Yue Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fanjin Meng, Chenyang Shao, Yuwei Yan, Qinglong Yang, Yiwen Song, Sijian Ren, Xinyuan Hu, Yu Li, Jie Feng, Chen Gao, and Yong Li. Towards large reasoning models: A survey of reinforced reasoning with large language models. *arXiv preprint arXiv:2501.09686*, 2025.
- [62] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [63] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujiu Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [64] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, 2018.
- [65] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

- [66] Anbang Ye, Qianran Ma, Jia Chen, Muqi Li, Tong Li, Fujiao Liu, Siqi Mai, Meichen Lu, Haitao Bao, and Yang You. Sop-agent: Empower general purpose ai agent with domain-specific sops. *arXiv preprint arXiv:2501.09316*, 2025.
- [67] Tik Yu Yim, Wenting Tan, Sum Yee Chan, Tak-Wah Lam, and Siu Ming Yiu. Asda: Automated skill distillation and adaptation for financial reasoning. *arXiv preprint arXiv:2603.16112*, 2026.
- [68] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [69] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026.
- [70] Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Experience compression spectrum: Unifying memory, skills, and rules in llm agents. *arXiv preprint arXiv:2604.15877*, 2026.
- [71] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- [72] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, pages 19632–19642, Vancouver, Canada, 2024.
- [73] Haiteng Zhao, Junhao Shen, Yiming Zhang, Songyang Gao, Kuikun Liu, Tianyou Ma, Fan Zheng, Dahua Lin, Wenwei Zhang, and Kai Chen. Achieving olympia-level geometry large language model agent via complexity boosting reinforcement learning. In *Proceedings of the 14th International Conference on Learning Representations*, Rio de Janeiro, Brazil, 2026.
- [74] Qingfei Zhao, Ruobing Wang, Dingling Xu, Daren Zha, and Limin Liu. R-search: Empowering llm reasoning with search via multi-reward reinforcement learning. *arXiv preprint arXiv:2506.04185*, 2025.
- [75] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.
- [76] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. Skillrouter: Skill routing for llm agents at scale. *arXiv preprint arXiv:2603.22455*, 2026.
- [77] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*, 2026.

The appendix is organized as below:

- Appendix A presents the local theoretical analysis that complements SLIM.
- Appendix B describes the detailed settings of SLIM and baselines.
- Appendix C describes the detailed experimental setup, including training configuration, retrieval settings, and lifecycle auditing details.
- Appendix D provides additional experimental results, including supplementary tables, curves, and robustness analyses.
- Appendix E introduces all prompts used in training, evaluation, and skill expansion.
- Appendix F summarizes the initial and expanded skill banks used by SLIM.
- Appendix G discusses the limitations of SLIM.
- Appendix H discusses the broader impacts of this research.

A Theoretical Analysis

This section provides the theoretical analysis that complements Section 4.1, Section 4.2, and Section 4.3. We keep the analysis local because Eq. (2) contains a continuous policy variable, a discrete active set, a conceptual black-box cost Ω , and a latent capacity constraint. The goal is not to prove global optimization of Eq. (2); instead, the lemmas give local sufficient conditions that explain why the lifecycle heuristics can be conservative under the stated operating assumptions. Throughout, skills outside the active external set may belong either to the latent internalized set $\mathcal{I}$ or to the inactive external set $\mathcal{U}$, so deactivation is not identified with internalization.

Let $F(\theta, \mathcal{A}) = \mathbb{E}_{x \sim \mathcal{X}}[\text{Perf}(x; \pi_\theta, \mathcal{A})]$ denote the expected task performance term in Eq. (2), and let $\mathcal{J}(\theta, \mathcal{A}) = F(\theta, \mathcal{A}) - \Omega(\mathcal{A})$ denote its performance-cost objective.

Assumption A.1. For a task x of type k , the hierarchical retriever in Eq. (3) preserves locally useful active task-specific skills with bounded miss probability. Specifically, if an active task-specific skill $s \in \mathcal{A}_t^k$ has task-conditioned marginal gain at least $\gamma_{\text{ret}} > 0$, then

$$\Pr(s \in \mathcal{Q}_t(x)) \geq 1 - \delta_{\text{ret}}. \quad (\text{A1})$$

This is the standard recall requirement when retrieval is used as a candidate-set reduction step, and is consistent with hierarchical skill libraries and skill routing systems [59, 76].

Lemma A.2. Under Assumption A.1, for any fixed locally useful active task-specific skill $s \in \mathcal{L}_t(x)$, task-conditioned retrieval misses s with probability at most δ_{ret} . Consequently, the probability of missing at least one skill in $\mathcal{L}_t(x)$ is at most $|\mathcal{L}_t(x)|\delta_{\text{ret}}$ by a union bound.

Proof. Let $\mathcal{L}_t(x) = \{s \in \mathcal{A}_t^k : s \text{ has task-conditioned marginal gain at least } \gamma_{\text{ret}} \text{ on } x\}$ denote the locally useful active task-specific skills for task x . For any $s \in \mathcal{L}_t(x)$, Assumption A.1 gives

$$\Pr(s \notin \mathcal{Q}_t(x)) = 1 - \Pr(s \in \mathcal{Q}_t(x)) \leq \delta_{\text{ret}}. \quad (\text{A2})$$

This proves the fixed-skill claim. For the set-level event,

$$\Pr(\exists s \in \mathcal{L}_t(x) : s \notin \mathcal{Q}_t(x)) \leq \sum_{s \in \mathcal{L}_t(x)} \Pr(s \notin \mathcal{Q}_t(x)) \leq |\mathcal{L}_t(x)|\delta_{\text{ret}}, \quad (\text{A3})$$

where the first inequality is the union bound. This completes the proof. $\square$

Assumption A.3. For a fixed active set $\mathcal{A}_t$, the GRPO objective is locally aligned with the performance term. If one GRPO update maps θ_t to θ_{t+1} , then

$$F(\theta_{t+1}, \mathcal{A}_t) - F(\theta_t, \mathcal{A}_t) \geq c_{\text{RL}}(J_{\text{GRPO}}(\theta_{t+1}; \mathcal{A}_t) - J_{\text{GRPO}}(\theta_t; \mathcal{A}_t)) - \varepsilon_{\text{RL}}, \quad (\text{A4})$$

where $c_{\text{RL}} > 0$ and $\varepsilon_{\text{RL}} \geq 0$ capture local surrogate mismatch. This is the standard local-improvement view used in PPO/GRPO-style policy optimization [12, 45, 68].

Assumption A.4. For an audited skill $s \in \mathcal{A}_t$, the smoothed leave-one-skill-out estimate concentrates around the population marginal external contribution under the updated policy θ_{t+1} . Let

$$\Delta_{\mathcal{X},t}(s) = F(\theta_{t+1}, \mathcal{A}_t) - F(\theta_{t+1}, \mathcal{A}_t \setminus \{s\}). \quad (\text{A5})$$

After EMA smoothing and sufficient exposure, the audit estimate satisfies

$$|\bar{\Delta}_t(s) - \Delta_{\mathcal{X},t}(s)| \leq \varepsilon_{\text{val}} \quad (\text{A6})$$

with probability at least $1 - \delta_{\text{val}}$. Across audit rounds, conditioned on the current policy and active set, validation errors are conditionally independent or satisfy an equivalent mixing condition. This is a finite-validation concentration condition for counterfactual ablation: bounded validation metrics admit concentration around their expectation under sufficient samples [21], while the patience and minimum-exposure conditions in Eq. (6) avoid decisions from a single noisy audit.

Assumption A.5. The lifecycle controller operates under a fixed physical budget. Due to strict limits on the maximum context window L_{max} , top- K routing slots, bounded skill artifact length, and finite audit budget, the marginal computational, routing, and context overhead of adding or removing one skill is strictly bounded. Therefore, for any single-skill move from $\mathcal{A}$ to $\mathcal{A}'$, there exists $B_{\text{op}} > 0$ such that

$$|\Omega(\mathcal{A}') - \Omega(\mathcal{A})| \leq B_{\text{op}}. \quad (\text{A7})$$

This statement is tied to the operating regime of the system rather than to a global model or estimator of Ω ; it is consistent with skill routing and long-context studies showing that routing and context costs are concrete system-level quantities [32, 76]. In the experiments, we report realized operational costs such as active skill count, audit calls, wall-clock time, and retrieval complexity rather than estimating the absolute value of Ω .

Lemma A.6. Under Assumption A.4, the smoothed leave-one-skill-out estimate $\bar{\Delta}_t(s)$ is an ε_{val} -accurate proxy for the true marginal external contribution $\Delta_{\mathcal{X},t}(s)$ with probability at least $1 - \delta_{\text{val}}$.

Proof. From Assumption A.4, the event

$$\mathcal{E}_{\text{val}}(s) = \{|\bar{\Delta}_t(s) - \Delta_{\mathcal{X},t}(s)| \leq \varepsilon_{\text{val}}\} \quad (\text{A8})$$

satisfies $\Pr(\mathcal{E}_{\text{val}}(s)) \geq 1 - \delta_{\text{val}}$. On $\mathcal{E}_{\text{val}}(s)$,

$$-\varepsilon_{\text{val}} \leq \bar{\Delta}_t(s) - \Delta_{\mathcal{X},t}(s) \leq \varepsilon_{\text{val}}, \quad (\text{A9})$$

which is equivalent to

$$\Delta_{\mathcal{X},t}(s) - \varepsilon_{\text{val}} \leq \bar{\Delta}_t(s) \leq \Delta_{\mathcal{X},t}(s) + \varepsilon_{\text{val}}. \quad (\text{A10})$$

Thus $\bar{\Delta}_t(s)$ is an ε_{val} -accurate proxy for $\Delta_{\mathcal{X},t}(s)$ with probability at least $1 - \delta_{\text{val}}$. This completes the proof. $\square$

Lemma A.7. Under Assumption A.5, for any single-skill move from $\mathcal{A}$ to $\mathcal{A}'$, where $\mathcal{A}' = \mathcal{A} \cup \{s\}$ or $\mathcal{A}' = \mathcal{A} \setminus \{s\}$, the absolute cost difference satisfies

$$|\Omega(\mathcal{A}') - \Omega(\mathcal{A})| \leq B_{\text{op}}. \quad (\text{A11})$$

Proof. This is exactly the operating-regime cost bound in Assumption A.5. For completeness, consider the two single-skill moves. If $\mathcal{A}' = \mathcal{A} \cup \{s\}$, then

$$|\Omega(\mathcal{A}') - \Omega(\mathcal{A})| = \Omega(\mathcal{A} \cup \{s\}) - \Omega(\mathcal{A}) \leq B_{\text{op}}, \quad (\text{A12})$$

where equality uses monotonicity of Ω and the inequality uses Assumption A.5. If $\mathcal{A}' = \mathcal{A} \setminus \{s\}$, then

$$|\Omega(\mathcal{A}') - \Omega(\mathcal{A})| = \Omega(\mathcal{A}) - \Omega(\mathcal{A} \setminus \{s\}) \leq B_{\text{op}}. \quad (\text{A13})$$

This completes the proof. $\square$

Lemma A.8. Suppose Assumption A.4 and Assumption A.5 hold. We analyze one accepted lifecycle move, while Section B.1 applies a strictly bounded number of moves per audit cycle as a local approximation analogous to batched policy updates. On the event $\mathcal{E}_{\text{val}}(s)$ in Lemma A.6, which holds with probability at least $1 - \delta_{\text{val}}$ for each audited skill, the lifecycle rules in Eq. (5)–Eq. (7) are conservative single-move decisions under the following sufficient margins. If $\tau_{\text{keep}} \geq B_{\text{op}} + \varepsilon_{\text{val}}$, Eq. (5) blocks a removal that cannot improve $\mathcal{J}$. If Eq. (6) is triggered and the saved cost satisfies $\Delta\Omega_t^-(s) \geq \tau_{\text{retire}} + \varepsilon_{\text{val}}$, retiring s improves $\mathcal{J}$. If Eq. (7) selects s_{new} whose expected performance gain is at least B_{op} , adding s_{new} does not decrease $\mathcal{J}$. For an audit that evaluates at most M_{audit} skills, the validation-dependent retain/retire conclusions hold jointly with probability at least $1 - M_{\text{audit}}\delta_{\text{val}}$ by a union bound.

Proof. For retain, the local objective change of removing s is

$$\mathcal{J}(\theta_{t+1}, \mathcal{A}_t \setminus \{s\}) - \mathcal{J}(\theta_{t+1}, \mathcal{A}_t) = -\Delta_{\mathcal{X},t}(s) + \Delta\Omega_t^-(s), \quad (\text{A14})$$

where $\Delta\Omega_t^-(s) = \Omega(\mathcal{A}_t) - \Omega(\mathcal{A}_t \setminus \{s\})$. If Eq. (5) is triggered, then $\bar{\Delta}_t(s) \geq \tau_{\text{keep}}$. Combining $\tau_{\text{keep}} \geq B_{\text{op}} + \varepsilon_{\text{val}}$ with Lemma A.6 gives

$$\Delta_{\mathcal{X},t}(s) \geq \bar{\Delta}_t(s) - \varepsilon_{\text{val}} \geq B_{\text{op}}. \quad (\text{A15})$$

By Lemma A.7, $\Delta\Omega_t^-(s) \leq B_{\text{op}}$. Therefore,

$$\begin{aligned} \mathcal{J}(\theta_{t+1}, \mathcal{A}_t \setminus \{s\}) - \mathcal{J}(\theta_{t+1}, \mathcal{A}_t) &= -\Delta_{\mathcal{X},t}(s) + \Delta\Omega_t^-(s) \\ &\leq -B_{\text{op}} + B_{\text{op}} = 0. \end{aligned} \quad (\text{A16})$$

Thus retaining s blocks a removal that cannot improve $\mathcal{J}$.

For retire, Eq. (6) gives $\bar{\Delta}_t(s) < \tau_{\text{retire}}$. By Lemma A.6,

$$\Delta_{\mathcal{X},t}(s) \leq \bar{\Delta}_t(s) + \varepsilon_{\text{val}} < \tau_{\text{retire}} + \varepsilon_{\text{val}}. \quad (\text{A17})$$

If $\Delta\Omega_t^-(s) \geq \tau_{\text{retire}} + \varepsilon_{\text{val}}$, then

$$\begin{aligned} \mathcal{J}(\theta_{t+1}, \mathcal{A}_t \setminus \{s\}) - \mathcal{J}(\theta_{t+1}, \mathcal{A}_t) &= -\Delta_{\mathcal{X},t}(s) + \Delta\Omega_t^-(s) \\ &> -(\tau_{\text{retire}} + \varepsilon_{\text{val}}) + \tau_{\text{retire}} + \varepsilon_{\text{val}} = 0. \end{aligned} \quad (\text{A18})$$

Thus the accepted retire move improves $\mathcal{J}$ in the single-move analysis under the stated saved-cost margin. Since $\Delta\Omega_t^-(s)$ is not directly observed by the algorithm, Eq. (6) should be interpreted as a conservative heuristic whose improvement guarantee holds only when the recovered external-support cost exceeds this sufficient margin.

For expand, let

$$G_t(s_{\text{new}}) = F(\theta_{t+1}, \mathcal{A}_t \cup \{s_{\text{new}}\}) - F(\theta_{t+1}, \mathcal{A}_t) \quad (\text{A19})$$

denote the expected performance gain of adding the new skill. The local objective change is

$$\begin{aligned} \mathcal{J}(\theta_{t+1}, \mathcal{A}_t \cup \{s_{\text{new}}\}) - \mathcal{J}(\theta_{t+1}, \mathcal{A}_t) &= G_t(s_{\text{new}}) - (\Omega(\mathcal{A}_t \cup \{s_{\text{new}}\}) - \Omega(\mathcal{A}_t)) \\ &\geq G_t(s_{\text{new}}) - B_{\text{op}}, \end{aligned} \quad (\text{A20})$$

where the inequality follows from Lemma A.7. If $G_t(s_{\text{new}}) \geq B_{\text{op}}$, the expand move does not decrease $\mathcal{J}$. The probability statement follows from Lemma A.6; applying the union bound over at most M_{audit} audited skills gives joint probability at least $1 - M_{\text{audit}}\delta_{\text{val}}$ for the validation-dependent retain/retire conclusions. This completes the proof. $\square$

Lemma A.9. Suppose Assumption A.4 holds across audit rounds. If an active skill s has true marginal external contribution $\Delta_{\mathcal{X},r}(s) \geq \tau_{\text{retire}} + \gamma$ for p consecutive audit rounds $r = t - p + 1, \dots, t$, where $\gamma > 0$, then there exists an effective concentration constant $c_{\text{eff}} > 0$, depending on validation sample size, EMA smoothing, and temporal mixing, such that the probability that s is falsely retired by the patience condition in Eq. (6) is at most $\exp(-c_{\text{eff}}p\gamma^2)$.

Proof. For a single audit round r , false low-contribution evidence requires $\bar{\Delta}_r(s) < \tau_{\text{retire}}$. Since $\Delta_{\mathcal{X},r}(s) \geq \tau_{\text{retire}} + \gamma$, this event implies

$$\bar{\Delta}_r(s) - \Delta_{\mathcal{X},r}(s) < -\gamma. \quad (\text{A21})$$

Although EMA introduces temporal correlation into $\bar{\Delta}_r(s)$, each audit round incorporates fresh validation samples. Under the mixing condition in Assumption A.4, the resulting smoothed estimate still satisfies an effective concentration bound. Therefore, by Hoeffding-type concentration for bounded validation metrics with effective sample size and mixing correction [21], there exists $c_{\text{eff}} > 0$ such that

$$\Pr(\bar{\Delta}_r(s) < \tau_{\text{retire}}) \leq \exp(-c_{\text{eff}}\gamma^2). \quad (\text{A22})$$

The patience condition requires this false low-contribution event to occur for p consecutive audit rounds. Under the conditional independence or equivalent mixing condition in Assumption A.4, the fresh validation samples across audit rounds yield an effective compounded concentration bound:

$$\Pr(\ell_t(s) \geq p) \leq \exp(-c_{\text{eff}}p\gamma^2). \quad (\text{A23})$$

Thus patience exponentially reduces the probability of falsely retiring a skill whose true marginal contribution remains above the retire threshold. This completes the proof. $\square$

Lemma A.10. Assume Assumption A.4. Define an active skill $s' \in \mathcal{A}_t$ as externally necessary under $(\theta_{t+1}, \mathcal{A}_t)$ if $\Delta_{\mathcal{X},t}(s') > 0$. If $\Delta_{\mathcal{X},t}(s') \geq \tau_{\text{retire}} + \varepsilon_{\text{val}}$, then $\bar{\Delta}_t(s') \geq \tau_{\text{retire}}$ with probability at least $1 - \delta_{\text{val}}$, so Eq. (6) cannot retire s' .

Proof. Since s' is externally necessary and satisfies $\Delta_{\mathcal{X},t}(s') \geq \tau_{\text{retire}} + \varepsilon_{\text{val}}$, removing its external support decreases expected performance by at least this margin. By Lemma A.6, with probability at least $1 - \delta_{\text{val}}$,

$$\bar{\Delta}_t(s') \geq \Delta_{\mathcal{X},t}(s') - \varepsilon_{\text{val}}. \quad (\text{A24})$$

If $\Delta_{\mathcal{X},t}(s') \geq \tau_{\text{retire}} + \varepsilon_{\text{val}}$, then

$$\begin{aligned} \bar{\Delta}_t(s') &\geq \Delta_{\mathcal{X},t}(s') - \varepsilon_{\text{val}} \\ &\geq \tau_{\text{retire}} + \varepsilon_{\text{val}} - \varepsilon_{\text{val}} \\ &= \tau_{\text{retire}}. \end{aligned} \quad (\text{A25})$$

Thus the first condition of Eq. (6), namely $\bar{\Delta}_t(s') < \tau_{\text{retire}}$, is false. Therefore s' cannot be retired by Eq. (6). Finite model capacity motivates why such externally necessary skills may persist, but the lemma itself only uses the observable marginal external contribution. It protects currently audited active skills and does not claim that previously retired inactive skills are immune to later forgetting. This completes the proof. $\square$

B Implementation Details

B.1 SLIM Setup

Backbone and Data Protocol. All main SLIM experiments use Qwen3-4B [63] as the policy model. We use the train split for GRPO updates, the validation split for lifecycle auditing and training-time monitoring, and the test split only for final reporting. For ALFWorld, we use 16 training tasks per update, 32 validation tasks per validation pass, a maximum prompt length of 4096, a maximum response length of 512, right truncation, and no overlength prompt filtering. For SearchQA, we use 64 training tasks per update, 512 validation tasks per validation pass, a maximum prompt length of 5000, a maximum response length of 700, left truncation, and overlength prompt filtering. ALFWorld runs for 120 GRPO steps and SearchQA runs for 180 GRPO steps both with validation every 10 steps. Validation and final evaluation use sampled generation with temperature 0.4.

Training. SLIM uses the GRPO objective Eq. (1) and does not use a separate warmup or cold-start stage. ALFWorld uses $n = 8$ rollouts per prompt, a maximum of 50 environment steps, and 50 turns of interaction history. SearchQA uses $n = 4$ rollouts per prompt, a maximum of 4 environment steps, 4 turns of history, and the shared search backend used by all SearchQA methods. The optimizer uses learning rate 10^{-6} in both benchmarks. The PPO mini-batch and per-device micro-batch sizes are 32 and 2 for ALFWorld, and 512 and 8 for SearchQA. Rewards are outcome-level environment rewards: completed successful trajectories receive the benchmark success reward and failed trajectories receive zero reward, while invalid-action penalties are applied during trajectory collection. The invalid-action penalty coefficient is 0.1 for ALFWorld and 0.01 for SearchQA. For the main SLIM runs, both actor-side KL loss and KL-in-reward regularization are disabled.

Retrieval. SLIM uses the hierarchical SkillRL-style skill bank described in Section 3. Task-specific skills are retrieved only from the active pool of the detected task type. The retrieval query is the current task description, while each skill key is a routing text concatenating the skill title, description or principle, when_to_apply field, body, tags, and task type. We embed both query and keys with Qwen3-Embedding-0.6B [71] and rank candidate skills by cosine similarity. We set the task-specific retrieval cap to $K = 3$ and the embedding threshold to $\tau_{\text{emb}} = 0.45$, so retrieval may insert fewer than three task-specific skills when no active skill is sufficiently relevant.

Skill Lifecycle. Lifecycle audit runs periodically after GRPO validation. Each audit records routed skills, validation outcomes, and routed failures under the current active set. In ALFWorld, each audit examines a bounded set of recently routed skills, including at most four task-specific skills selected among skills that appeared in the top- K retrieved set. In SearchQA, the same logic is used with a larger audit budget of at most 12 active skills because validation batches are larger and episodes are shorter. Missing candidates are skipped rather than replaced by extra candidates outside the audit budget. This

budget limits audit cost, while the exposure and patience conditions below prevent rarely routed skills from being retired solely because they were not frequently audited. For each audited skill s , SLIM computes the leave-one-skill-out marginal external contribution $\Delta_t(s)$ and updates the smoothed estimate $\bar{\Delta}_t(s)$ with EMA coefficient 0.9. Retirement uses $\tau_{\text{retire}} = 0.001$ and patience $p = 3$ in both benchmarks. The minimum exposure threshold is $n_{\text{min}} = 30$ for ALFWorld and $n_{\text{min}} = 20$ for SearchQA. Retain decisions use $\tau_{\text{keep}} = 0.03$ for ALFWorld and $\tau_{\text{keep}} = 0.05$ for SearchQA. Thus, a skill is retired only after it has been routed often enough and its smoothed contribution remains negligible for multiple audits. Expansion is task-specific only and is performed during training audits, never during final test evaluation. ALFWorld uses $\tau_{\text{expand}} = 0.40$, $n_{\text{expand}} = 20$, and creates at most two new skills per audit. SearchQA uses $\tau_{\text{expand}} = 0.40$, $n_{\text{expand}} = 15$, and creates at most three new skills per audit. A failure bucket is formed from validation failures routed to the same active skill and task type. The new skill inherits the task type of this bucket, so expansion adds support only to the corresponding task-specific pool. New skills are standalone SKILL.md artifacts generated by an OpenAI-compatible o3 skill-creator backbone using an Anthropic-style skill-creator workflow [7]. The expansion prompt, shown in Figure A11, includes representative failed tasks, summarized failure traces, and the insufficient active skills. We deduplicate against existing skills by title, trigger description, and embedding similarity of the routing text. We reject generated skills that are generic, duplicate an existing skill, are too short to define a useful workflow, or attempt to create new general or foundational skills.

Audit cost. Leave-one-skill-out auditing adds validation-time overhead, but it is bounded by the audit schedule and candidate budget. Audits run every 10 GRPO steps and use limited validation subsets. In ALFWorld, each audit evaluates at most one general-skill group and $M = 4$ task-specific skills; in SearchQA, the larger validation batch and shorter episodes allow at most 12 audited active skills. Thus, SLIM does not rerun validation for the full skill bank. In our runs, ALFWorld completes in about 20 hours and SearchQA completes in about 25 hours, which is the same order as Skill0 and SkillRL under the shared training stack. Table A7 in Appendix D.6 provides the detailed overhead comparison.

Inference. For skill-conditioned final inference, SLIM uses the final active set after training. Before each rollout, the agent inserts active general skills and retrieves task-specific skills using the same task-type-scoped embedding retrieval as in training. Final evaluation does not perform retain, retire, or expand operations. The environment prompts are shown in Figures A2 and A4, and the skill insertion format is shown in Figure A5.

Compute resources. From logged token counts and step timings, the ALFWorld full SLIM run is on the order of 10^{19} floating-point operations. SearchQA uses shorter episodes but a larger validation batch and longer training horizon, giving an overall compute budget on the order of 10^{19} – 10^{20} floating-point operations. These estimates include rollout generation, log-probability evaluation, policy updates, validation, and lifecycle audit reruns, but exclude one-time data preprocessing.

B.2 Baselines Setup

Zero-Shot and Few-Shot Prompting. The prompt-only baselines evaluate the backbone model without RL updates. Zero-shot prompting directly uses the benchmark interaction prompt in Figure A1 or Figure A3. Few-shot prompting prepends solved examples following Figure A6. The with-skill variants use the same prompt protocol but additionally insert retrieved external skills using Figure A5. Prompt baselines are evaluated with the same environment parser, success metric, and final test split as the RL methods, using batch size 16 for ALFWorld and 32 for SearchQA.

ReAct. ReAct is implemented as an environment-aligned reasoning-and-acting adapter under the shared evaluation stack. It uses the instruction in Figure A7, preserves the same action tags and parser as the main environment, and does not use persistent memory. This is a prompt-level adaptation of the ReAct idea rather than an upstream-exact execution with a separate tool interface.

Reflexion. Reflexion stores short reflections from failed trajectories and retrieves relevant reflections before later actions. The acting and reflection-generation prompts are shown in Figure A8. We retrieve at most three reflections and keep a bounded memory of 200 items. Generation uses deterministic service decoding with maximum 768 tokens, temperature 0, and top- $p = 1.0$. Our implementation preserves the failure-reflection-reuse mechanism under the shared environment protocol, but does not reproduce the original multi-trial training pipeline exactly.

ExpeL. ExpeL is implemented as an experience-lesson baseline. After each episode, the method distills a compact reusable lesson from the trajectory and retrieves at most three relevant lessons for future decisions, with the same 200-item memory cap and deterministic service decoding as Reflexion. Figure A9 gives the acting and lesson-distillation prompts. This adapter preserves the core experience-distillation idea while using the same action parser, success metric, and train/validation/test protocol as the other baselines.

Mem0. Mem0 is implemented as a lightweight long-term memory baseline. It extracts up to three short atomic memories after each episode, stores them in the same bounded 200-item memory bank, and retrieves at most three relevant memories before action generation, following Figure A10. This adaptation measures the value of compact retrieved memory under the shared evaluation stack rather than differences in external memory infrastructure.

GRPO and GRPO with Skills. The GRPO baseline uses the same RL optimizer, environment stack, and reward definition as SLIM, but does not receive external skill or memory context and does not use a warmup stage. ALFWorld uses train batch size 16, validation batch size 32, rollout count 8, maximum episode length 50, learning rate 10^{-6} , PPO mini-batch size 32, micro-batch size 2, 120 training steps, and validation every 10 steps. SearchQA uses train batch size 64, validation batch size 512, rollout count 4, maximum episode length 4, learning rate 10^{-6} , PPO mini-batch size 512, micro-batch size 8, 180 training steps, and validation every 10 steps. Both settings disable actor-side KL loss and KL-in-reward regularization, and use invalid-action penalty coefficients 0.1 for ALFWorld and 0.01 for SearchQA. GRPO with skills keeps the same optimizer and rollout settings but inserts retrieved external skills using the skill-conditioned prompt format in Figure A5; it does not perform retain, retire, or expand operations.

EvolveR. EvolveR is evaluated as an experience-lifecycle retrieval baseline. It distills past trajectories into reusable principles, retrieves relevant experience before rollout, and updates the experience store during training. We adapt EvolveR to the same ALFWorld and SearchQA environments, success-rate metric, and train/validation/test protocol. Where experience distillation or update requires a service model, we use the same OpenAI-compatible service family as the other method-style baselines. The adapter retrieves the top-3 experience principles before each rollout. The comparison therefore preserves the experience-driven lifecycle idea while keeping action parsing, reward computation, validation protocol, and service-model access aligned with the other methods. EvolveR does not estimate leave-one-skill-out marginal external contribution and does not maintain retain/retire/expand lifecycle states over skill artifacts.

Skill0. Skill0 is adapted as a scheduled skill-withdrawal baseline and does not use a separate warmup stage in our comparison. During training, the policy is exposed to external skills and the visible skill set is progressively reduced according to the curriculum schedule. ALFWorld uses the schedule [6, 3, 0] with train batch size 16, validation batch size 32, rollout count 8, maximum episode length 50, learning rate 10^{-6} , PPO mini-batch size 32, micro-batch size 2, 120 training steps, and validation every 10 steps. SearchQA uses the schedule [5, 3, 0] with train batch size 64, validation batch size 512, rollout count 4, maximum episode length 4, learning rate 10^{-6} , PPO mini-batch size 512, micro-batch size 8, and 180 training steps. Both settings disable actor-side KL loss and KL-in-reward regularization, and use the same invalid-action penalty coefficients as SLIM. The original Skill0 codebase includes text-rendering and OCR-related components, but our ALFWorld and SearchQA settings are text-native. We therefore disable OCR/text rendering to avoid introducing a visual-rendering confound; the original Skill0 comparison reports less than a three-point gain from text rendering, so this adaptation does not affect the main skill-lifecycle comparison. Skill0 is designed to approach zero-skill inference.

SkillRL. SkillRL is adapted as a persistent skill-augmented RL baseline and does not use a separate warmup stage in our comparison. The policy retrieves external skills during training and inference with the same SkillRL-style retrieval path used to initialize our skill bank, using the skill-conditioned prompt format in Figure A5. ALFWorld uses top-6 skill retrieval, dynamic skill-bank update with threshold 0.4, at most three new skills per update, train batch size 16, validation batch size 32, rollout count 8, maximum episode length 50, learning rate 10^{-6} , PPO mini-batch size 32, micro-batch size 2, 120 training steps, and validation every 10 steps. Its skill author/updater is a comparable OpenAI-compatible o3 backbone that analyzes failed trajectories and emits new JSON skill records, matching the SkillRL-style skill-bank format rather than producing SKILL.md artifacts. SearchQA uses the same top-6 skill retrieval and dynamic skill-bank update protocol, with train batch size

64, validation batch size 512, rollout count 4, maximum episode length 4, learning rate 10^{-6} , PPO mini-batch size 512, micro-batch size 8, and 180 training steps. Unlike GRPO, SkillRL retains a reference-policy KL loss, using KL coefficients 0.01 for ALFWorld and 0.001 for SearchQA, while KL-in-reward remains disabled. The original SkillRL implementation does not define the same explicit validation-stage protocol used in our experiments, so we add validation on the dedicated validation split and reserve the test split for final reporting. This adaptation preserves SkillRL’s core identity as persistent external skill augmentation.

B.3 Additional Baselines Setup

GPT-4o and Gemini-2.5-Pro on ALFWorld. For the closed-source ALFWorld comparison, we report the GPT-4o and Gemini-2.5-Pro results from SkillRL [59]. These results are used only in the expanded ALFWorld table in Appendix D.5 to contextualize the scale of agent performance under strong proprietary models. They are not used for hyperparameter selection, lifecycle auditing, or any training-time comparison.

SimpleMem on ALFWorld. SimpleMem is evaluated as a long-term memory baseline following its semantic memory compression and retrieval design [31]. Each completed ALFWorld episode is converted into a compact textual memory record containing the task description, key observations, actions, outcome, and a short lesson. Memories are indexed with Qwen3-Embedding-0.6B to match the embedding backbone used elsewhere in our experiments. Following the original SimpleMem configuration style, we enable planning and reflection during retrieval, use semantic retrieval as the primary route, and allow keyword and structured filters as auxiliary routes. The memory retriever considers up to 25 semantic candidates, 5 keyword candidates, and 5 structured candidates, then inserts at most three compact memories into the action prompt to keep the prompt budget comparable to the other memory baselines. The policy model is Qwen3-4B, and the environment parser, success metric, train/validation/test split, and final evaluation protocol are the same as the other baselines.

RLOO on ALFWorld. RLOO is evaluated as an RL optimizer baseline using the same ALFWorld environment stack as GRPO. The only optimizer-level change is the advantage estimator: for each prompt, we sample $K = 8$ rollouts and compute the leave-one-out advantage $\hat{A}_i = R_i - \frac{1}{K-1} \sum_{j \neq i} R_j$, following the REINFORCE leave-one-out formulation [1]. All other settings are matched to the ALFWorld GRPO baseline, including train batch size 16, validation batch size 32, maximum episode length 50, learning rate 10^{-6} , 120 training steps, validation every 10 steps, and no cold-start SFT. RLOO does not receive external skills, memory context, or lifecycle updates.

MemRL on ALFWorld. MemRL is evaluated as a runtime memory-reinforcement baseline that updates episodic memory without updating model parameters [69]. We preserve the original method structure by using proceduralization for memory construction, query-based retrieval, and adjustment-based memory updates. To keep the comparison fair, the policy model is Qwen3-4B and the embedding model is Qwen3-Embedding-0.6B, while the ALFWorld parser, few-shot examples, task split, and success metric are shared with the rest of our evaluation. The memory retriever uses $k_{\text{retrieve}} = 5$, at most 8 extracted keywords, add-similarity threshold 0.90, novelty threshold 0.85, unknown-detection threshold 0.62, and a value-aware candidate set of size 3. The value update follows the original single-step setting with learning rate $\alpha = 0.3$, discount $\gamma = 0$, success reward 1.0, failure reward -1.0 , and equal weights 0.5/0.5 for similarity and memory value in the combined retrieval score.

RAG on SearchQA. RAG is evaluated as a one-shot retrieval-augmented generation baseline on SearchQA [26]. Following the Search-R1 setting [23], each question is used as the retrieval query, the shared search corpus is queried before generation, and the top three retrieved passages are inserted into the prompt. The model then generates a final answer without iterative search calls, RL updates, skill retrieval, or lifecycle control. This baseline uses the same Qwen3-4B backbone, answer-matching rule, and test split as the other SearchQA methods.

Search-o1 on SearchQA. Search-o1 is evaluated as an inference-time agentic search baseline [27]. The model is prompted to reason, issue search queries when needed, read retrieved evidence, and continue reasoning before producing the final answer. We use the same search interface, top-3 retrieved passages per search call, maximum action budget of 4 search rounds, and answer evaluator as the SearchQA environment. Search-o1 does not update model parameters and does not use external skills; it differs from RAG mainly by allowing iterative search and reasoning during inference.

Search-R1 on SearchQA. Search-R1 is evaluated as an RL-with-search baseline following the Search-R1 training protocol [23]. During rollout, the policy alternates between model-generated reasoning tokens and search calls, and retrieved tokens are masked from the policy-gradient loss so that optimization is applied only to model-generated tokens. We use the shared SearchQA search interface with top-3 retrieved passages, maximum action budget 4, outcome exact-match reward, learning rate 10^{-6} , and the same train/validation/test separation as SLIM. This baseline does not use external skills or lifecycle operations, so it isolates the effect of learning to interact with search from the effect of managing a skill bank.

SFT on SearchQA. SFT is evaluated as a supervised fine-tuning baseline on SearchQA [11]. Training examples are constructed only from the training split using the same response format as the search-enabled environment. The model is optimized to imitate the target reasoning-and-answer trajectory, while validation is used only for checkpoint selection. SFT does not use outcome-reward RL, external skills, lifecycle auditing, or test trajectories during training.

Reject Sampling on SearchQA. Reject Sampling follows the Search-R1 baseline construction [2, 23]. For each training prompt, we sample five search-enabled candidate trajectories with the same search interface and action budget, keep trajectories whose final answer is correct under exact match, and fine-tune the model on the selected trajectories. This preserves the multi-turn LLM–search interaction format while replacing online RL with filtered supervised learning. Validation is used for checkpoint selection, and the final test split is held out until reporting.

C Evaluation Setup

Benchmark Protocol. We evaluate all methods on the two agent benchmarks used in the main paper. ALFWorld [50] is a long-horizon text-interaction benchmark in which the agent must complete household tasks by issuing admissible text actions. We report both the overall success rate and task-type success rates for Pick, Look, Clean, Heat, Cool, and Pick2, following the task categories used by the environment. Although ALFWorld has a fixed set of task types, RL supervision is collected over multi-step action trajectories, so each episode contributes many action-level decisions under the shared rollout protocol. SearchQA follows the search-augmented question-answering setting of Search-R1 [23]. The agent interacts with a search tool and must eventually output a final answer for questions from NQ [25], TriviaQA [24], PopQA [35], HotpotQA [64], 2Wiki [20], MuSiQue [51], and Bamboogle [42]. All methods use the same environment-side action parser, search interface, trajectory termination rule, and success evaluator within each benchmark.

Data Splits and Data Usage. We use explicit training, development, and final-evaluation partitions for both benchmarks. For ALFWorld, the training split contains 16 text-interaction tasks, the development split contains 64 tasks, and the final-evaluation split contains 128 tasks. For SearchQA, the training partition contains 169,615 questions from HotpotQA (90,447) and NQ (79,168). The development partition contains 4,000 examples and is balanced by skill type rather than by source: compare, direct retrieval, entity-attribute lookup, and multi-hop reasoning each contain 1,000 examples. Its source distribution is HotpotQA (1,246), PopQA (1,000), TriviaQA (743), 2Wiki (729), NQ (257), MuSiQue (24), and Bamboogle (1). The skill-type construction maps compare to 2Wiki/HotpotQA/MuSiQue/Bamboogle, direct retrieval to NQ/TriviaQA, entity-attribute lookup to PopQA, and multi-hop reasoning to HotpotQA. The final-evaluation partition contains 51,713 examples from PopQA (14,267), 2Wiki (12,576), TriviaQA (11,313), HotpotQA (7,405), NQ (3,610), MuSiQue (2,417), and Bamboogle (125). For SearchQA, the development partition is organized as a fixed validation view over the benchmark source distribution. The protocol is fixed and shared across all methods: policy optimization uses the training partition, lifecycle auditing and checkpoint selection use the development view, and final reporting uses the final-evaluation partition after the policy checkpoint and active skill set are frozen.

Success Metrics. The primary metric in all tables is success rate. In ALFWorld, a rollout is successful if the environment returns the terminal success signal for the target household objective. We compute the overall ALFWorld success rate by averaging the binary success indicators over evaluation episodes, and compute task-type success rates by grouping episodes according to their ALFWorld task type. In SearchQA, a rollout is successful if the final answer emitted by the agent matches the benchmark ground-truth answer under the shared answer-matching rule used by the SearchQA environment. We report per-source success rates and their average over the seven SearchQA sources listed above.

Robustness Diagnostics. Following common practice in LLM RL studies, the main training curves are reported from a single run under a fixed training protocol because full agentic RL training is computationally expensive. To mitigate variance concerns, we report category-level results, lifecycle trajectories, ablation studies, audit overhead, and transfer or initialization sensitivity diagnostics. These analyses suggest that the gains are not explained by a single favorable lifecycle trajectory.

Final Evaluation Protocol. For trainable methods, final evaluation is run from the selected checkpoint with all training-time adaptation disabled. For SLIM, the final active skill set is fixed before test evaluation; test rollouts may retrieve from this frozen active set, but retain, retire, expand, lifecycle audit, and skill creation are disabled. For Skill0, the curriculum state is fixed by its schedule before final evaluation. For SkillRL, the skill bank and retrieval configuration are fixed before test evaluation. Prompt-based and agent- or memory-based baselines are also evaluated on the same final test split; when a method accumulates memory during evaluation, that memory is created only from earlier evaluation episodes of that method and is not shared across methods.

Fairness Controls. All main comparisons use Qwen3-4B [63] as the backbone model. Trainable methods start from the same base checkpoint and use the same benchmark environment, reward definition, action format, and success metric. Prompt-based and method-style baselines use the same served backbone and the same environment wrappers, so their generated actions are parsed and judged by the same environment-side parser as RL methods. We match prompt-length limits, response-length limits, rollout horizons, validation cadence, and final test splits whenever the method class permits it; method-specific differences such as Skill0’s scheduled skill withdrawal, SkillRL’s persistent skill retrieval, and SLIM’s lifecycle audit are kept because they define the corresponding algorithms. Fairness is especially important for methods with skill expansion or skill-bank updates. Whenever methods involve comparable modules, we instantiate those modules with the same implementation choices. With-skill methods use the same initial SkillRL-style skill bank family, the same skill insertion format shown in Figure A5, and the same embedding retrieval backbone where retrieval is required. Methods that create or update skills use comparable OpenAI-compatible creator or updater backbones under their own method-specific output formats; in particular, SLIM expansion and SkillRL skill-bank updates both use an OpenAI-compatible o3 backbone, while EvolveR and memory-style baselines use the same service-model family where experience or memory construction requires one. Methods that do not define skill creation are not given extra generated skills, and no method creates or updates skills from test data. Thus, the comparison changes the lifecycle policy, not the strength of the underlying retrieval, prompting, environment, or skill-generation infrastructure. Regardless of whether a method is run through the RL training stack or through a service-based evaluation wrapper, generation is served with SGLang. Unless otherwise specified by a method-specific protocol, we keep the LLM inference configuration at the shared default setting and enable sampled decoding for evaluation.

No Cold-Start SFT. All RL baselines are evaluated under the same controlled no-warmup setting. Specifically, all RL-based methods start from the same base checkpoint without cold-start SFT or warmup, so the comparison isolates online RL and lifecycle behavior rather than supervised pre-adaptation. This avoids confounding lifecycle effects with a policy that has already been trained to follow a particular skill format, trajectory style, or action pattern before RL begins. We tune method-specific hyperparameters only on the validation split.

Leakage Prevention. The key separation in our protocol is that training-time control signals are never computed from the final test split. In particular, SLIM estimates marginal external contribution, routed failure buckets, retain/retire decisions, and expansion triggers only on the validation split, never on test tasks. Expansion prompts are constructed strictly from validation-side audit failures; they never include final-evaluation prompts, trajectories, failures, labels, or answers. Expanded skills are created before final evaluation, and the skill creator never receives test prompts, test trajectories, test failures, or test labels. During final evaluation, the policy checkpoint and active skill set are frozen; test rollouts may retrieve from the frozen active set, but they cannot create, edit, retire, or expand skills. Skill0 and SkillRL are evaluated under the same train/validation/test separation: validation may affect training-time curriculum, skill-bank updates, or monitoring where the method requires it, but final test examples are held out until reporting. For online memory baselines, any memory accumulated during final evaluation is private to that baseline and arises only from earlier episodes in the same sequential evaluation run; it is not used to select checkpoints, update model parameters, tune prompts, or modify skill banks before evaluation. This prevents test labels, test trajectories, and test failures from entering policy optimization, skill-bank updates, lifecycle decisions, or skill expansion.

Table A1: Cross-source generalization on SearchQA. The training split contains NQ and HotpotQA questions, while the held-out sources include TriviaQA, PopQA, 2Wiki, MuSiQue, and Bamboogle. Train-source and held-out averages are macro averages over the listed sources; Overall Avg. is the benchmark micro average. **Best** results are highlighted.

Method	Train-source			Held-out sources					Held-out Avg.	Overall Avg.
	NQ	HotpotQA	Avg.	TriviaQA	PopQA	2Wiki	MuSiQue	Bamboogle		
GRPO	35.9	30.8	33.4	57.8	36.5	30.1	9.2	30.4	32.8	37.5
SkillRL [†]	36.8	31.5	34.2	59.8	36.9	29.7	10.3	31.2	33.6	38.1
Skill0	37.9	32.7	35.3	59.5	38.6	31.9	10.3	32.8	34.6	39.3
SLIM	38.6	37.2	37.9	62.2	40.0	31.7	12.8	37.6	36.9	41.0
SLIM [†]	38.4	36.9	37.7	62.1	40.4	31.5	12.7	36.0	36.5	41.0

Table A2: Transfer evaluation of the final active skill bank learned by SLIM. “None” uses no external skill, “Initial skill bank” uses the pre-training skill bank, and “Final SLIM active skills” uses the final active set learned by SLIM. The transferred skill bank is fixed and no lifecycle update is performed during evaluation.

Prompting Method	Skill Source	ALFWorld Avg.	SearchQA Avg.
Zero-Shot	None	41.4	32.3
	Initial skill bank	63.3	31.5
	Final SLIM active skills	65.8	32.7
Few-Shot	None	39.1	35.5
	Initial skill bank	64.1	34.8
	Final SLIM active skills	66.9	35.8

D Additional Experimental Results

This appendix provides supplementary experiments that support the main claims in Section 6. We focus on six questions that are not fully covered by the main table: whether the learned lifecycle transfers across SearchQA task families, whether the final active skill bank is useful beyond the trained policy, whether SLIM is robust to different initial skill banks, whether the reported gains are robust, how selected additional baselines compare on each benchmark separately, and how much audit overhead lifecycle management introduces.

D.1 Cross-Task Generalization on SearchQA

SearchQA provides a natural cross-source generalization setting because training uses HotpotQA and NQ, while the final test file also includes TriviaQA, PopQA, 2Wiki, MuSiQue, and Bamboogle. Table A1 re-organizes the main SearchQA results into train-source and held-out-source subsets. SLIM obtains the best train-source average and the best held-out average, improving the held-out average from 34.6 for Skill0 to 36.9. The gains are broad rather than concentrated in one dataset: SLIM is strongest on TriviaQA, MuSiQue, and Bamboogle, while SLIM[†] is strongest on PopQA. This supports that lifecycle-guided training learns reusable QA procedures instead of only adapting to the sources observed during RL training.

D.2 Transfer of the Final Active Skill Bank

Table A2 transfers different skill sources to policies that did not participate in lifecycle training: no external skills, the initial skill bank before RL, and the final active skill set learned by SLIM. On ALFWorld, the transfer is strong: final SLIM skills improve zero-shot and few-shot policies by 24.4 and 27.8 points over no-skill prompting, and by 2.5 and 2.8 points over the initial skill bank. This suggests that the learned active set captures reusable procedural guidance rather than only serving as a private scaffold for the trained SLIM policy. SearchQA shows a different pattern. The final active skills improve zero-shot and few-shot prompting only slightly over no-skill prompting, by 0.4 and 0.3 points, but they consistently outperform the initial skill bank by 1.2 and 1.0 points. Thus, on SearchQA the final skill bank mainly filters noisy or less useful external guidance rather than

Table A3: Sensitivity to the initial skill bank. The table reports final performance and final lifecycle statistics under different initialization conditions.

Initial Skill Bank	Final Avg.	Final Active Skills	Retired Skills	Expanded Skills
Empty skill bank	76.4	18	8	26
Weak initial skill bank	81.2	23	15	29
Noisy initial skill bank	85.6	25	46	33
Original initial skill bank	87.5	21	33	16

Table A4: Bootstrap robustness of SLIM gains over the strongest skill-based baselines. Gaps are success-rate percentage points. Confidence intervals are computed with 10,000 independent aggregate bootstrap resamples over reconstructed binary test outcomes.

Benchmark	Comparison	Mean Gap	95% CI	Crosses 0?
ALFWorld	SLIM [†] – Skill0	+13.3	[+3.9, +22.7]	No
ALFWorld	SLIM [†] – SkillRL [†]	+12.6	[+3.1, +21.9]	No
SearchQA	SLIM [†] – Skill0	+1.72	[+1.13, +2.31]	No
SearchQA	SLIM [†] – SkillRL [†]	+2.95	[+2.35, +3.52]	No

providing large direct transfer gains. This supports the broader view that skill-bank transferability depends on whether the benchmark benefits from persistent external procedural support.

D.3 Sensitivity to Skill Initialization

Robustness to initialization. This experiment varies the initial skill bank while keeping the training and audit protocol fixed. Table A3 shows that SLIM is robust to imperfect initial banks because expansion can repair missing coverage and retirement can filter noisy or low-value skills. At the same time, the original initial bank still gives the highest final score, indicating that lifecycle management complements rather than replaces skill initialization.

Expansion from weak coverage. Starting from an empty skill bank, SLIM reaches 76.4% and creates 26 skills during training. This shows that SLIM is not merely selecting from a fixed library; it can build useful external support from persistent failure cases. However, the 11.1-point gap to the original setting indicates that expansion from scratch does not fully replace a reasonably informative initial library. With only 25% of the original skills, SLIM improves to 81.2% and expands 29 new skills, suggesting that the lifecycle controller can recover part of the missing task coverage when initialization is incomplete.

Filtering noisy skills. The noisy setting provides the strongest robustness evidence. Even when 30% of original skills are corrupted and 30% extra mismatched skills are injected, SLIM reaches 85.6%, only 1.9 points below the original setting. The controller retires 46 skills and expands 33 new ones, indicating that SLIM actively removes harmful external knowledge and repairs missing coverage rather than blindly preserving the initial bank.

Initialization still matters. The original skill bank gives the best result, but SLIM still substantially reshapes it by retiring 33 skills and expanding 16 new ones, leaving a compact active set of 21 skills. Thus, the gain does not come from static reuse of the initial skills. A reasonable initial bank improves the ceiling, while lifecycle management determines which skills should remain active after RL.

D.4 Robustness of SLIM Performance

Table A4 reports an independent aggregate bootstrap analysis over test outcomes. All confidence intervals remain above zero. On ALFWorld, the intervals are wider because the test set contains 128 episodes, but the lower bounds remain positive against both Skill0 and SkillRL[†]. On SearchQA, the absolute gains are smaller, but the large test set yields tight intervals, showing that the improvement is small but statistically reliable under this resampling check.

Table A5: Expanded ALFWorld comparison. All entries report task success rate. [†] denotes evaluation with retrieved external skills. * denotes closed-source model results copied from SkillRL [59]. Avg. denotes micro average. **Best** results are highlighted.

Method	Pick	Look	Clean	Heat	Cool	Pick2	Avg.
GPT-4o*	75.3	60.8	31.2	56.7	21.6	49.8	48.0
Gemini-2.5-Pro*	92.8	63.3	62.1	69.0	26.6	58.7	60.3
SimpleMem	100.0	25.0	9.5	53.3	39.1	11.8	48.7
MemRL	100.0	33.3	10.0	46.2	9.5	6.7	41.3
RLOO	85.3	100.0	50.0	75.0	52.2	38.9	65.3
SLIM [†]	92.9	100.0	91.4	78.3	88.5	81.2	87.5

Table A6: Expanded SearchQA comparison. All entries report success rate. [†] denotes evaluation with retrieved external skills. Avg. denotes micro average. **Best** results are highlighted.

Method	NQ	TriviaQA	PopQA	HotpotQA	2Wiki	MuSiQue	Bamboogle	Avg.
RAG	42.1	65.7	46.4	37.1	30.7	8.4	28.0	37.6
Search-o1	30.6	51.8	22.6	28.2	38.2	11.2	41.6	33.3
Search-R1	38.0	59.7	38.4	35.7	40.1	13.2	35.2	37.2
SFT	48.7	49.4	43.6	28.8	31.4	7.1	32.0	34.4
Reject Sampling	38.0	61.2	40.0	35.1	31.6	14.3	37.6	36.8
SLIM [†]	38.4	62.1	40.4	36.9	31.5	12.7	36.0	41.0

D.5 Expanded Results

Additional ALFWorld baselines. The expanded ALFWorld comparison adds closed-source, memory-based, and RL-based baselines beyond the main table. SLIM remains competitive against these broader alternatives while using the same environment and evaluation protocol, which strengthens the conclusion that lifecycle management improves procedural agent training rather than only outperforming a narrow baseline set.

Additional SearchQA baselines. The expanded SearchQA comparison includes retrieval-augmented, supervised, rejection-sampling, and RL-style baselines. These results show that SLIM is not merely benefiting from a stronger prompting or search interface; its gains remain consistent when compared with alternative ways of improving search-augmented QA behavior.

D.6 Audit Overhead Comparison

SLIM introduces leave-one-skill-out audit calls, but the audit budget is periodic and capped. Table A7 shows that SLIM is more expensive than ordinary validation, but its lifecycle cost is bounded by K rather than the full skill-bank size. In contrast, SkillRL has lighter validation but can accumulate a growing skill bank, while Skill0 is cheaper but mainly models progressive withdrawal. Although Ω is not instantiated during training, its operational effects are reflected by measurable quantities. SLIM reduces the final active skill count from 38 initial skills plus 16 expansions to 21 active skills, while SkillRL grows to 73. Its audit budget is capped and does not scale with the full skill bank, and wall-clock remains comparable to SkillRL and Skill0. Together with the w/o Retirement and Fixed Active Set Size ablations, these results indicate that SLIM’s gains are not due to unbounded external support or simple prompt-budget control.

E Prompts

This section provides the prompt templates used for environment interaction, skill insertion, baseline adapters, and skill expansion. We keep placeholders such as {task_description} and {retrieved_memories} explicit so that the templates can be mapped directly to the training and evaluation protocol.

Table A7: Audit overhead comparison on the ALFWorld training setting. V denotes one ordinary validation pass, S is the full skill-bank size, S_k is the task-specific active pool for task type k , and K is the bounded SLIM audit budget.

Method	Additional validation-equivalent calls	Skill-bank scaling	Bottleneck	Main benefit
Skill0	$O(1)$ coarse curriculum comparisons	Shrinking skill set	Curriculum validation	Efficient withdrawal toward zero-skill inference
SkillRL	$O(1)$ ordinary validation	$O(S)$ retrieval / maintenance as skills accumulate	Growing skill bank and prompt management	Persistent skill augmentation and exploration support
SLIM	$O(1 + K)$, $K \leq 5$	$O(S_k)$ retrieval and bounded audit	Periodic leave-one-skill-out audit	Dynamic external boundary through retain, retire, and expand

Environment interaction prompts. Figure A2 and Figure A4 show the skill-conditioned rollout prompts used for ALFWorld and SearchQA. Figure A1 and Figure A3 show the corresponding no-skill templates.

Skill insertion format. Figure A5 shows how retrieved general and task-specific skills are inserted into the agent context.

Baseline prompts. Figure A6, Figure A7, Figure A8, Figure A9, and Figure A10 provide the prompts used by prompt-based and memory-style baselines.

Skill creation prompt. Figure A11 gives the expansion prompt used by SLIM to create new standalone task-specific SKILL.md artifacts from routed failures.

F Skill Bank Details

We summarize representative skills from the hierarchical skill banks used by SLIM. The tables include general skills and task-specific skills for ALFWorld and SearchQA, with concise trigger conditions and one-line procedural content. Dynamically expanded skills are inserted into the task-specific pool of the corresponding task type and become eligible for later retrieval and lifecycle auditing.

G Limitations

SLIM has three main limitations. First, marginal external contribution is a local single-skill leave-one-out estimate conditioned on the current policy, routing behavior, and active set. It is not a global Shapley-style attribution and does not capture high-order interactions among skills. Second, lifecycle thresholds and audit budgets require validation tuning, so transferring the same configuration to substantially different domains may require additional calibration. Third, lifecycle auditing remains practical in our current setting but may become expensive for very large skill banks, where more scalable audit candidate selection would be needed.

H Broader Impacts

This work studies how external capabilities should be allocated between model parameters and modular skill artifacts during agentic RL. By making capability allocation explicit, SLIM may improve the controllability, auditability, and sample efficiency of skill-based agentic RL, while also offering a clearer view of which behaviors are better internalized and which are better preserved externally. We hope this perspective will support future research on more adaptive, interpretable, and practically effective agent training paradigms.

Title: ALFWorld No-Skill Rollout Prompt

You are an expert agent operating in the ALFRED Embodied Environment. Your task is to: {task_description}
Prior to this step, you have already taken {step_count} step(s). Below are the most recent {history_length} observations and the corresponding actions you took: {action_history}
You are now at step {current_step} and your current observation is: {current_observation}
Your admissible actions of the current situation are: [{admissible_actions}].
Now it's your turn to take an action. You should first reason step-by-step within <think> </think> tags, then choose one admissible action within <action> </action> tags.

Figure A1: ALFWorld no-skill rollout prompt.

Title: ALFWorld Skill-Conditioned Rollout Prompt

You are an expert agent operating in the ALFRED Embodied Environment. Your task is to: {task_description}
{retrieved_memories}
Prior to this step, you have already taken {step_count} step(s). Below are the most recent {history_length} observations and the corresponding actions you took: {action_history}
You are now at step {current_step} and your current observation is: {current_observation}
Your admissible actions of the current situation are: [{admissible_actions}].
Now it's your turn to take an action. You should first reason step-by-step within <think> </think> tags, then choose one admissible action within <action> </action> tags.

Figure A2: ALFWorld skill-conditioned rollout prompt. The placeholder {retrieved_memories} is filled by the skill insertion format in Figure A5.

Title: SearchQA No-Skill Rollout Prompt

You are an expert agent tasked with answering the given question step-by-step.
 Your question: {task_description}
 Prior to this step, you have already taken {step_count} step(s). History: {memory_context}
 Now it's your turn to respond. First reason within <think> </think> tags.
 Then choose one action: search using <search> your query </search>, or answer using <answer> final answer </answer>.

Figure A3: SearchQA no-skill rollout prompt.

Title: SearchQA Skill-Conditioned Rollout Prompt

You are an expert agent tasked with answering the given question step-by-step.
 Your question: {task_description}
 {retrieved_memories}
 Prior to this step, you have already taken {step_count} step(s). History: {memory_context}
 Now it's your turn to respond. First reason within <think> </think> tags.
 Then choose one action: search using <search> your query </search>, or answer using <answer> final answer </answer>.

Figure A4: SearchQA skill-conditioned rollout prompt.

Title: Retrieved Skill Insertion Format

GENERAL SKILLS ###
 Here are some useful general strategies for solving the task effectively:
 1. {title}: {principle} Apply this when {when_to_apply}
 ...
 ### TASK: {task_type} ###
 For {readable_task} tasks, apply these specific strategies:
 1. {title}: {principle} Apply this when {when_to_apply}
 ...

Figure A5: Skill insertion format used by SLIM and skill-conditioned baselines. General skills are inserted as a group when active, while task-specific skills are retrieved by task type and semantic similarity.

Title: Zero-Shot and Few-Shot Prompting

Zero-shot:
 {obs_text}
 Few-shot prefix:
 Below are solved examples. Follow the same output format and decision style.
 [Example 1] User: {example_user} Assistant: {example_assistant}
 [Example 2] User: {example_user} Assistant: {example_assistant}
 [Current Task]
 {obs_text}

Figure A6: Zero-shot and few-shot prompting templates. Skill-conditioned variants insert the retrieved skill block from Figure A5 into the current task context.

Title: ReAct Prompt

System: You are following the ReAct paradigm: interleave explicit reasoning with environment actions. Reason concisely but concretely. Always output reasoning inside `<think>...</think>` and the single next action inside the required action tags.

User: {obs_text}

Figure A7: ReAct adapter prompt used under the shared environment protocol.

Title: Reflexion Prompt

System: You are using the Reflexion style. Before acting, consult prior reflective notes about common mistakes. Avoid repeating known errors. Keep the output format exact.

User: Past reflections from related mistakes:

- {reflection_1}
- {reflection_2}

{obs_text}

Reflection generation: Write one short actionable reflection that would help avoid the same failure next time.

Figure A8: Reflexion adapter prompt and reflection-generation instruction.

Title: ExpeL Prompt

System: You are using the ExpeL style: consult reusable lessons distilled from previous trajectories, then act using the current environment state.

User: Retrieved experience lessons:

- {lesson_1}
 - {lesson_2}
- {obs_text}

Lesson distillation: Distill one compact reusable lesson from this trajectory. It should be transferable and phrased as an actionable rule.

Figure A9: ExpeL adapter prompt and lesson-distillation instruction.

Title: Mem0 Prompt

System: You are using a Mem0-style external memory. Retrieve small factual or procedural memories from prior tasks and use them only when relevant.

User: Relevant memories from prior tasks:

- {memory_1}
 - {memory_2}
- {obs_text}

Memory extraction: Extract up to 3 short atomic memories from this trajectory and return them as a JSON list of strings.

Figure A10: Mem0-style memory prompt and memory-extraction instruction.

Title: SLIM Skill Creation Prompt

System: Create one new standalone task-specific SKILL.md artifact that addresses the dominant failure pattern. Do not rewrite an existing skill and do not create a general or foundational skill.

User: Task type: {task_type}

Failed tasks and traces: {failure_bucket}

Currently selected skills that were insufficient: {active_skills}

Output a skill with:

1. Skill name
2. Routing-oriented description
3. Trigger condition
4. Concise procedural workflow

Reject generic names, duplicate titles, overly short workflows, and any output that targets general skills.

Figure A11: Skill creation prompt used by SLIM during expansion. The prompt follows an Anthropic-style skill-creator workflow and produces new task-specific SKILL.md artifacts.

Table A8: Representative ALFWorld skills used by SLIM, including the skills analyzed in the lifecycle case study.

Task	Skill ID	Skill Name	Trigger	Content
General	gen_004	Track Counts & Progress	Multi-instance goals such as putting two objects.	Maintain a counter of remaining goal objects and stop only when the count reaches zero.
General	gen_011	Efficient Relation Search	Goals mentioning both a target and reference object.	Search one object near the other instead of treating them independently.
General	gen_001	Systematic Exploration	Goal object count is unmet and unexplored locations remain.	Search plausible surfaces or containers once before revisiting checked locations.
General	gen_002	Immediate Acquisition	First visual confirmation of a goal-relevant object.	Take a required object as soon as it becomes visible and reachable.
General	gen_003	Destination First Policy	Holding a goal object after identifying its target location.	Navigate directly to the receptacle and place the object before resuming search.
pick_and_place	pic_001	Systematic First-Pass Search	Before acquiring required objects.	Maintain a checklist of visible and closed candidates and inspect each once.
pick_and_place	pic_002	Grab When Seen	First sight of an unheld target object.	Immediately take a needed visible object before moving elsewhere.
clean	cle_003	Sink First for Cleaning	Target object is held and must be clean.	Go to the nearest sink or basin and clean the object before placement.
clean	cle_005	State Verification Before Drop	After cleaning and before final placement.	Verify the object is clean; clean again if the state is uncertain.
heat	hea_003	Open Then Heat	At the microwave with the target object held.	Open the microwave, place the object inside, and execute the heat action.
cool	coo_005	Direct Post-Cooling Delivery	Cooling action succeeds.	Deliver the cooled object directly to the destination without detours.
cool	coo_002	Confirm Object Match	After acquiring an object in a cooling task.	Verify that the held item matches the requested target type; otherwise drop it and resume search.
cool	coo_004	Enforce Cooling Before Placement	Holding the correct object before final placement.	Do not place the target object until a fridge or freezer cooling action has succeeded.
cool	dyn_verify_cooling_completion	Verify Cooling Completion	After placing an object inside a cooling appliance.	Confirm the object is cool before retrieval, then proceed directly to delivery.

Table A9: Representative SearchQA skills used by SLIM.

Task	Skill ID	Skill Name	Trigger	Content
General	gen_001	Decompose Then Search	Complex or multi-part questions.	Break the question into minimal sub-questions and search each before synthesis.
General	gen_002	Precision Query Crafting	Initial query formulation.	Use exact entity names and target attributes while avoiding filler words.
General	gen_003	Iterative Query Refinement	First result set lacks definitive evidence.	Add qualifiers, alternate names, dates, or context instead of repeating the same query.
General	gen_004	Source-Backed Assertions	Before committing to a final answer.	Answer only after locating supporting evidence.
General	gen_005	Cross-Check Multiple Sources	Facts may be outdated or disputed.	Validate names, dates, and numbers against multiple independent sources.
direct_retrieval	dir_001	Isolate Core Query	Start of direct retrieval.	Strip the question to the key entity and sought fact, then search that pair.
direct_retrieval	dir_002	Refine When Empty	Initial search gives weak or no hits.	Reformulate with synonyms, alternate names, dates, or quoted phrases.
direct_retrieval	dir_003	Anchor With Quotes	Distinctive phrases, lyrics, titles, or quotes.	Wrap unique phrases in quotation marks to retrieve exact-match sources.
direct_retrieval	dyn_direct_retrieval_search_first_answer_second	Search First, Answer Second	Factoid or definition queries.	Always run an external search before relying on memory.
multi_hop_reasoning	mul_001	Decompose Question First	Multi-hop questions linking multiple facts.	Split the question into explicit sub-questions before searching.
multi_hop_reasoning	mul_003	Collect-Then-Compare	Comparative multi-hop tasks.	Retrieve concrete values for all items before comparing.
entity_attribute_lookup	ent_001	Direct Attribute Query	Full, unambiguous entity name is given.	Include the full entity name and target attribute in the first search.
entity_attribute_lookup	ent_003	Two-Source Cross-Check	First plausible answer appears or attribute is uncertain.	Confirm the attribute in at least two independent sources.
compare	com_001	Decompose & Isolate	Reading a comparison-type question.	Split the question into entities and the single attribute to compare.
compare	com_003	Normalize Before Comparing	After gathering each entity's attribute.	Convert values to a common comparable form before judging equality or order.